

C. Attention and Transformers

C.1 Attention

C.2 Transformer

where the attention mechanism first appeared and why it was introduced



Tiam Heydari, PhD
Postdoctoral Fellow, UBC

Part I: Birth of attention Attention

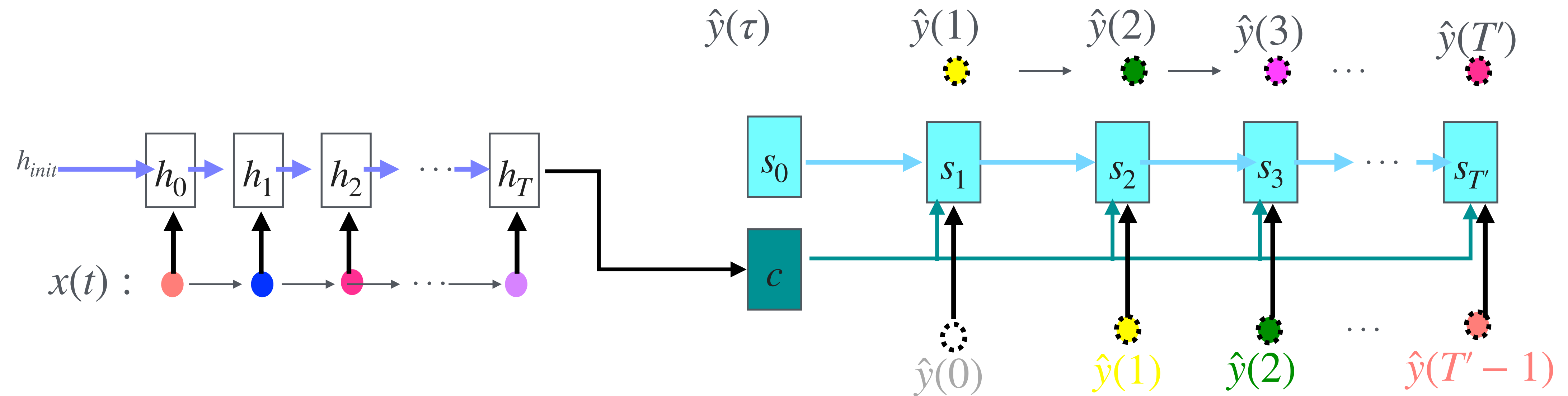
Sequence to sequence tasks via RNNs (e.g translation):

Encoder reads the full input sequence

Final encoder state becomes one context vector **c**

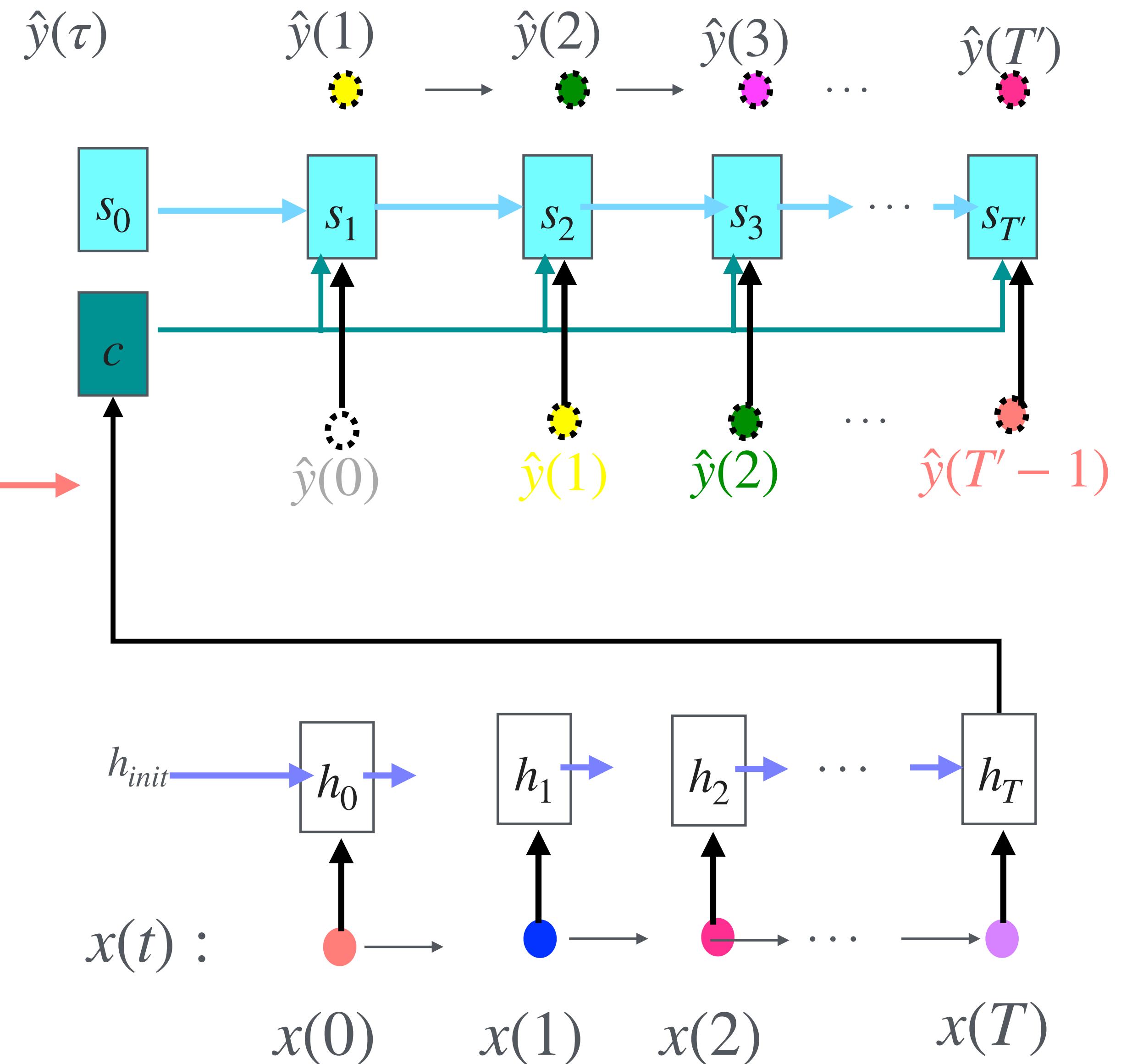
Decoder uses the same **c** at every output step

Long inputs create an information **bottleneck**



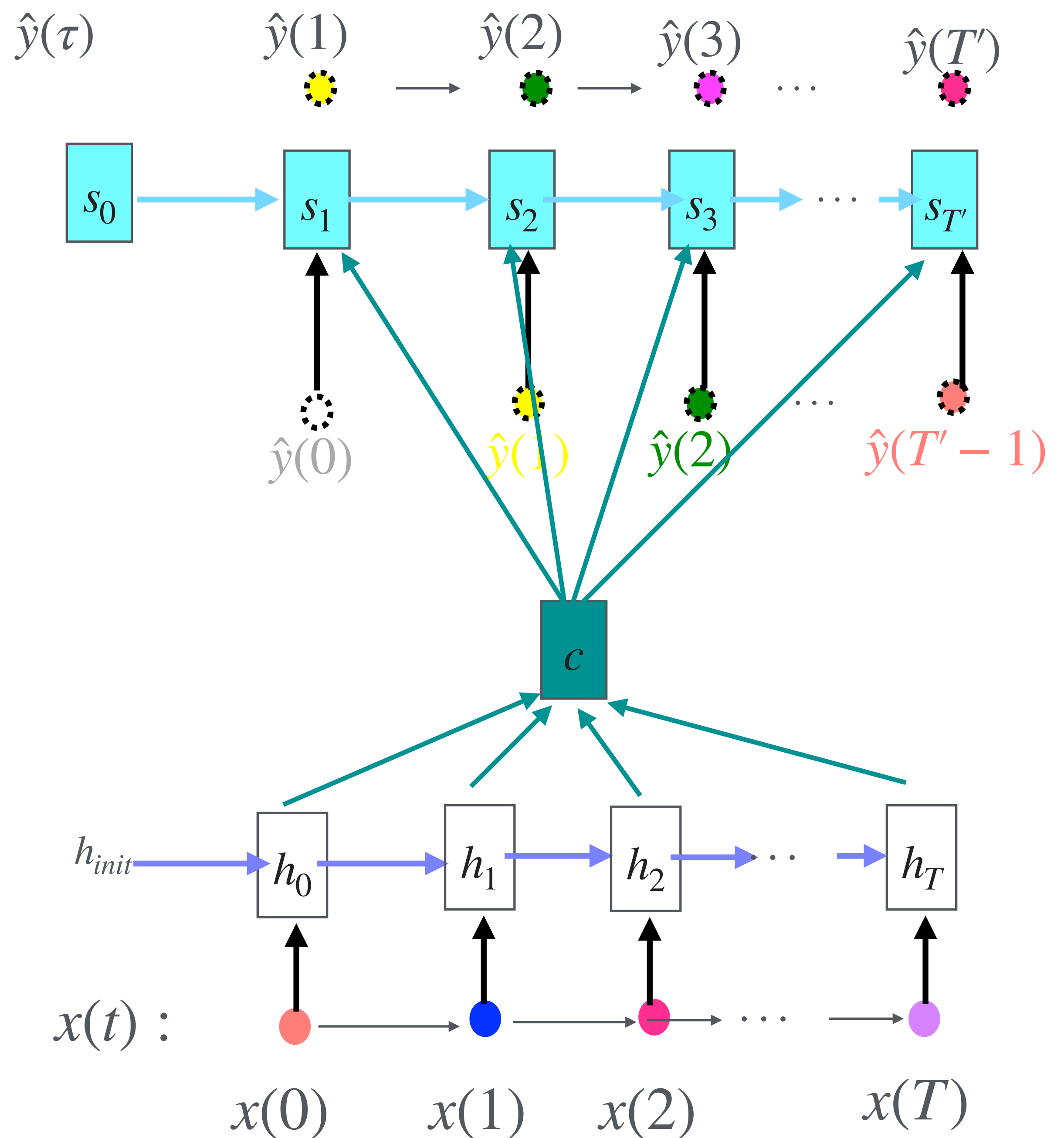
Sequence to sequence tasks via RNNs (e.g translation):

- This means a large amount of information must be squeezed into one vector, c
- This introduces a serious **bottleneck problem**
- For very long sequences, such as DNA or a book, etc, this becomes unrealistic



Idea 1: context sees everything

- Let $\mathbf{c}$ use all encoder states, not only the final state
- But every decoder step still receives the same $\mathbf{c}$
- Problem reappears: The full input is still compressed into one vector
- Even if $\mathbf{c}$ depends on all encoder states ($h_1, \dots, h_T$), it still must compress them into a single vector for all decoder steps!



Idea 2: A context for each decoder step

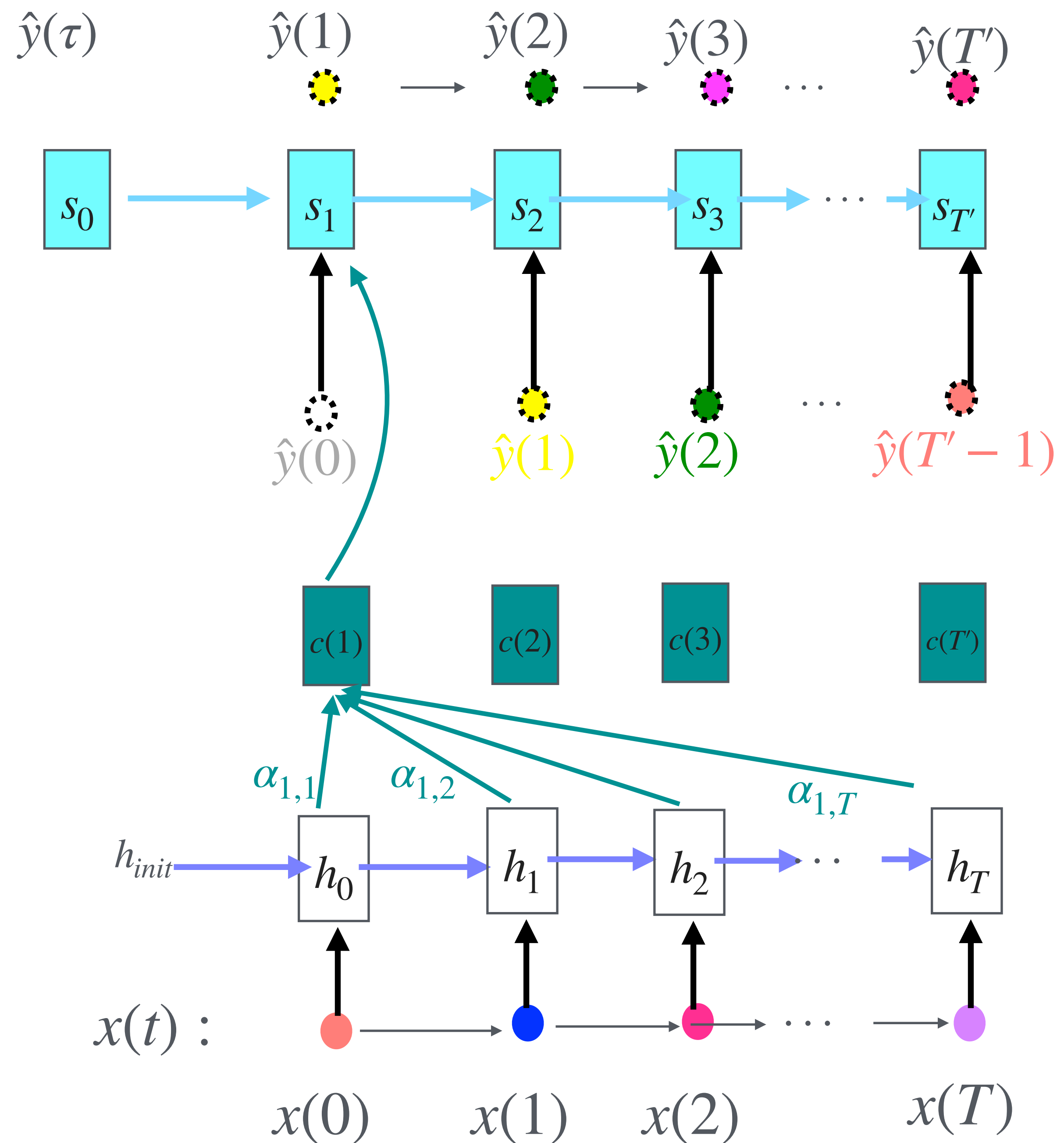
- Each decoder step gets its own context vector:

$$c(i) = \sum_j \alpha_{i,j} h_j$$

$\alpha_{i,j}$ controls how much decoder step i uses encoder state h_j

Basically, each context vector, does something simple, it calculate a weighted sum of all hidden states of the encoder

$\alpha_{i,j}$ for now can just be learnable weights(basically scalar values...)



Idea 2: A context for each decoder step

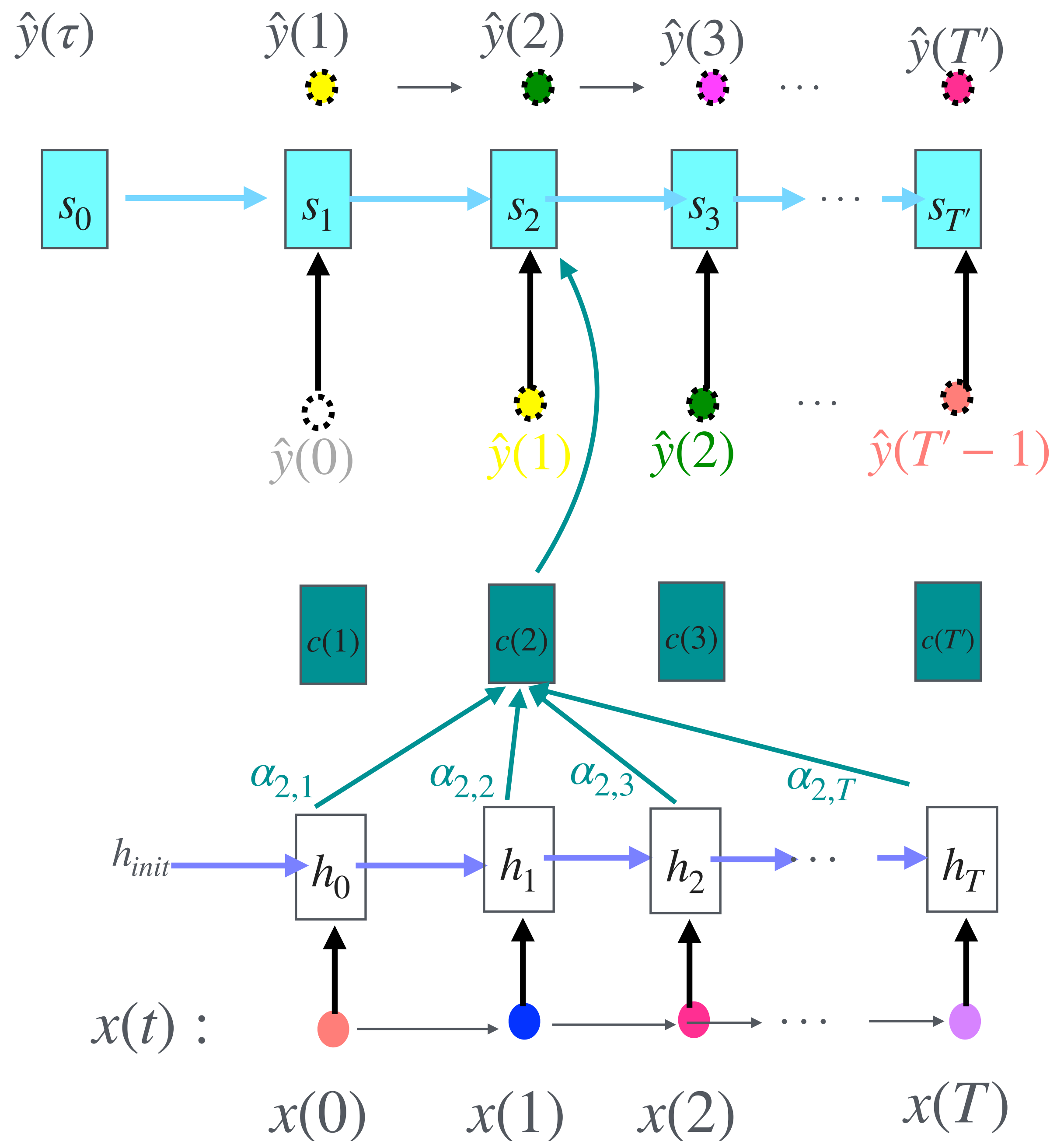
- Each decoder step gets its own context vector:

$$c(i) = \sum_j \alpha_{i,j} h_j$$

$\alpha_{i,j}$ controls how much decoder step i uses encoder state h_j

Basically, each context vector, does something simple, it calculate a weighted sum of all hidden states of the encoder

$\alpha_{i,j}$ for now can just be learnable weights(basically scalar values...)



Idea 2: A context for each decoder step

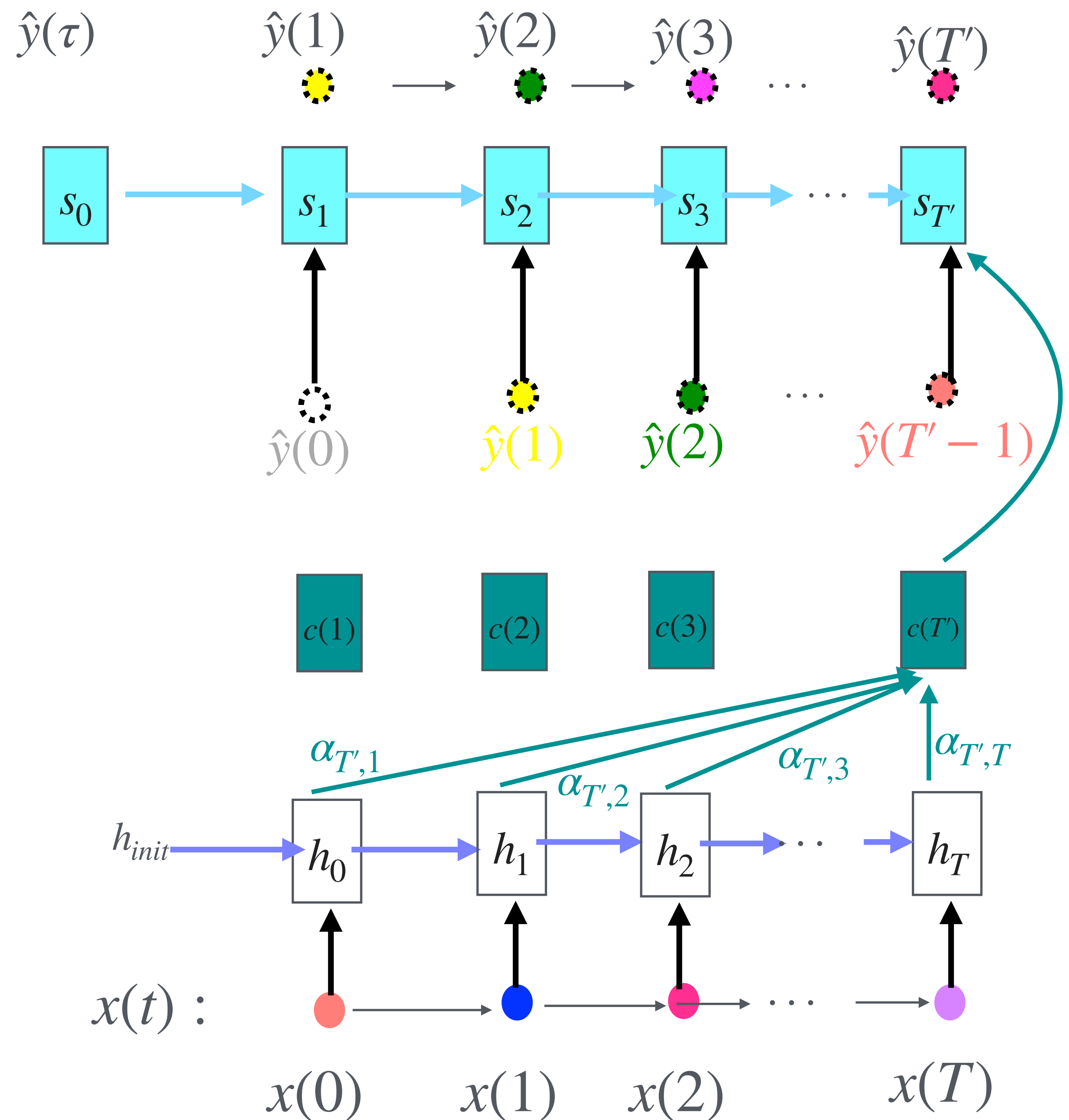
- Each decoder step gets its own context vector:

$$c(i) = \sum_j \alpha_{i,j} h_j$$

$\alpha_{i,j}$ controls how much decoder step i uses encoder state h_j

Basically, each context vector, does something simple, it calculate a weighted sum of all hidden states of the encoder

$\alpha_{i,j}$ for now can just be learnable weights (basically scalar values...)

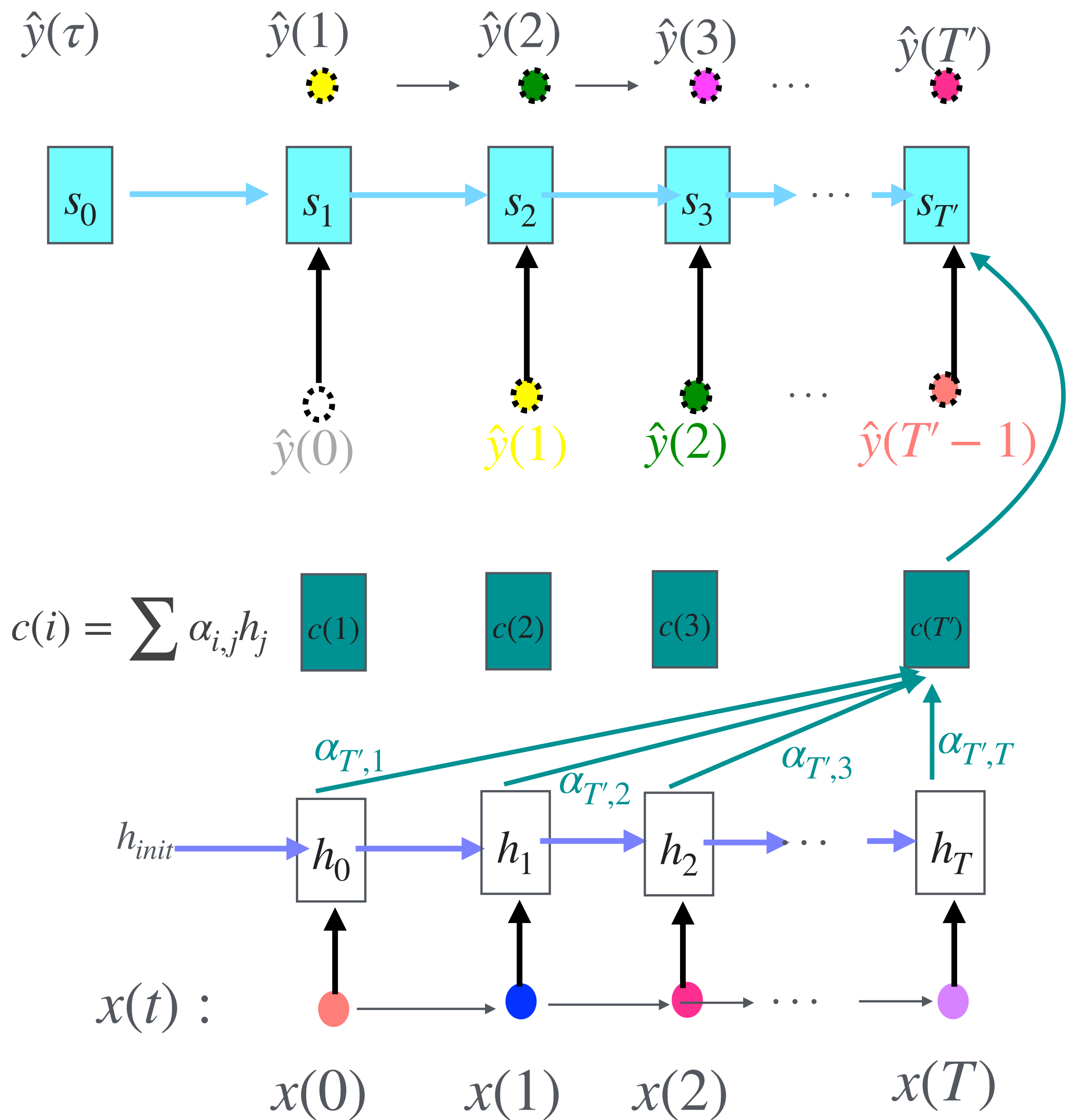


Idea 3: A computable context per step

- But there is still **a problem**
- Fixed $\alpha_{i,j}$ would be the same for every input sequence.
- But what decoder step i needs depends on:

current decoder state s_{i-1} and encoder state h_j

“So the weights $\alpha_{i,j}$ should **be computed dynamically**, not fixed in advance by training”



Idea 3: A computable context per step

- Make the weights depend on the current decoder state and each encoder state:

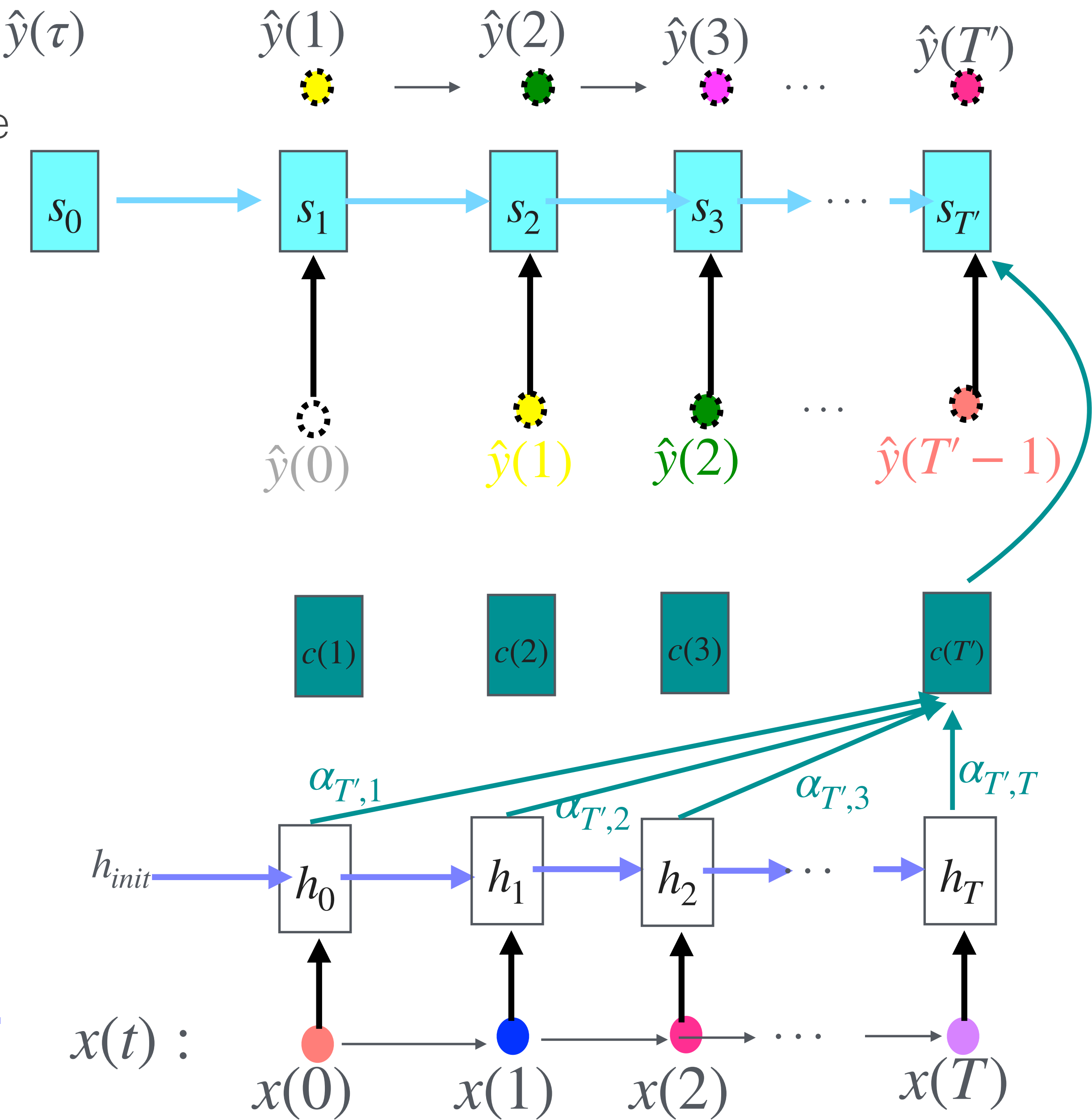
$$\alpha_{i,j} = F(s_{i-1}, h_j)$$

$$c(i) = \sum \alpha_{i,j} h_j$$

- What we want from function F :

- (a) larger score $\Rightarrow$ more influence
- (b) All encoder positions should compete with each other
- (c) The weights should be comparable across positions
- (d) the final context should stay on a controlled scale

competition, comparability, controlled scale.



Idea 3: A computable context per step

Attention: score $\rightarrow$ normalize $\rightarrow$ combine

1. Score each encoder state for decoder step i :

$$e_{i,j} = F(s_{i-1}, h_j)$$

2. Normalize scores across encoder positions:

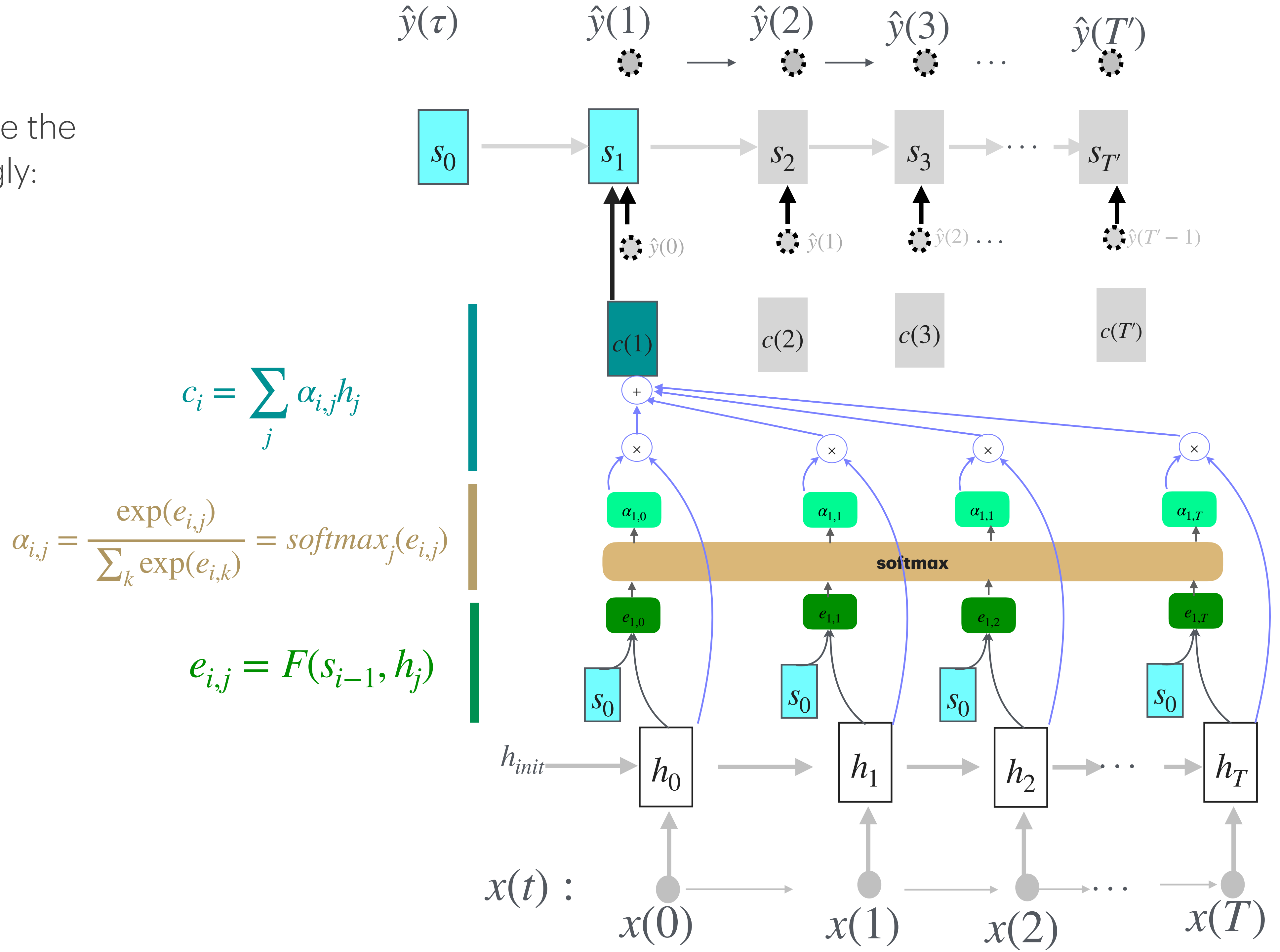
$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_k \exp(e_{i,k})} = \text{softmax}_j(e_{i,j})$$

3. Combine encoder states into a context vector:

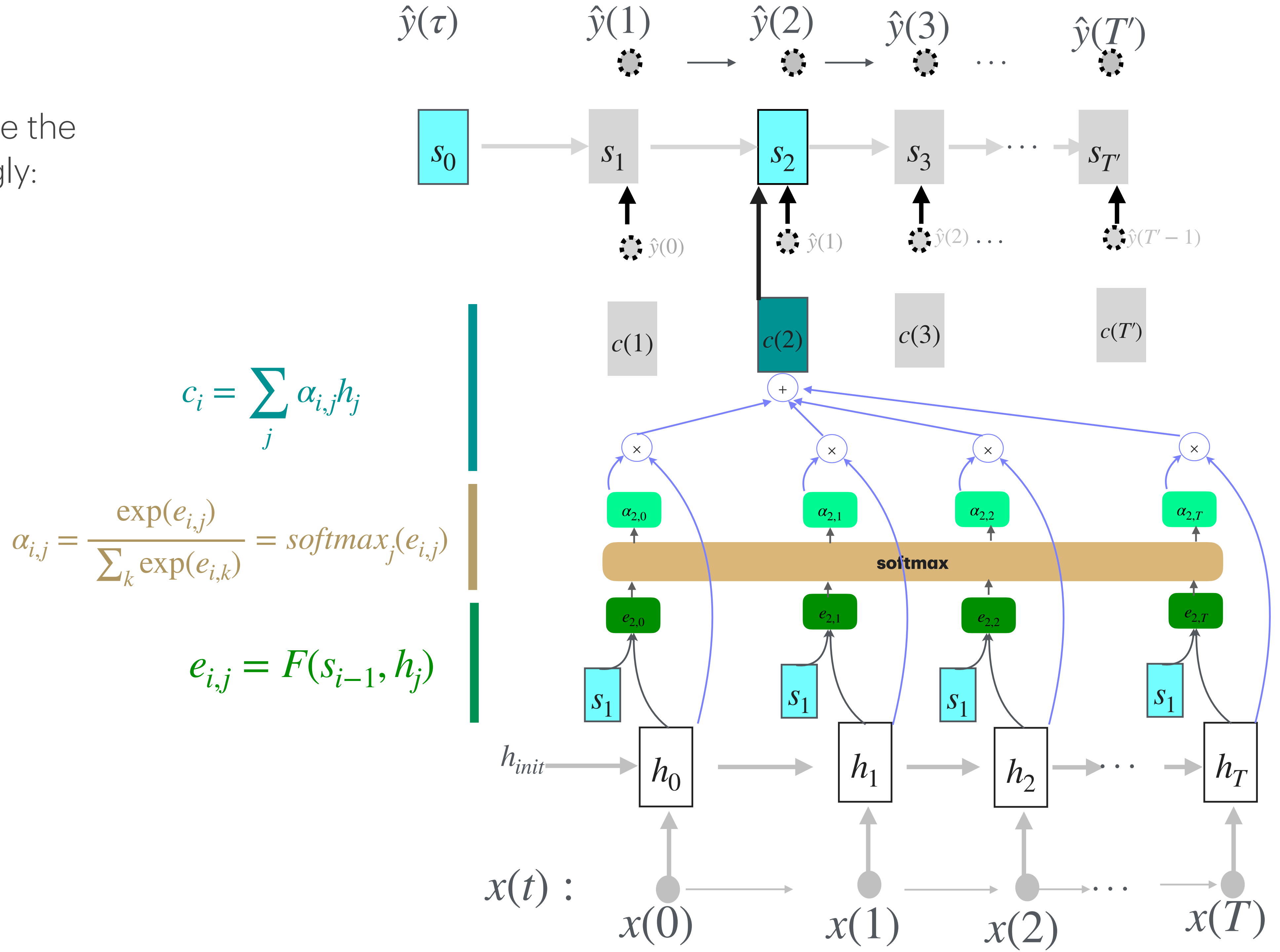
$$c^{(i)} = \sum_j \alpha_{i,j} h_j$$

Note that: $\alpha_{i,j} \geq 0$, $\sum_j \alpha_{i,j} = 1$

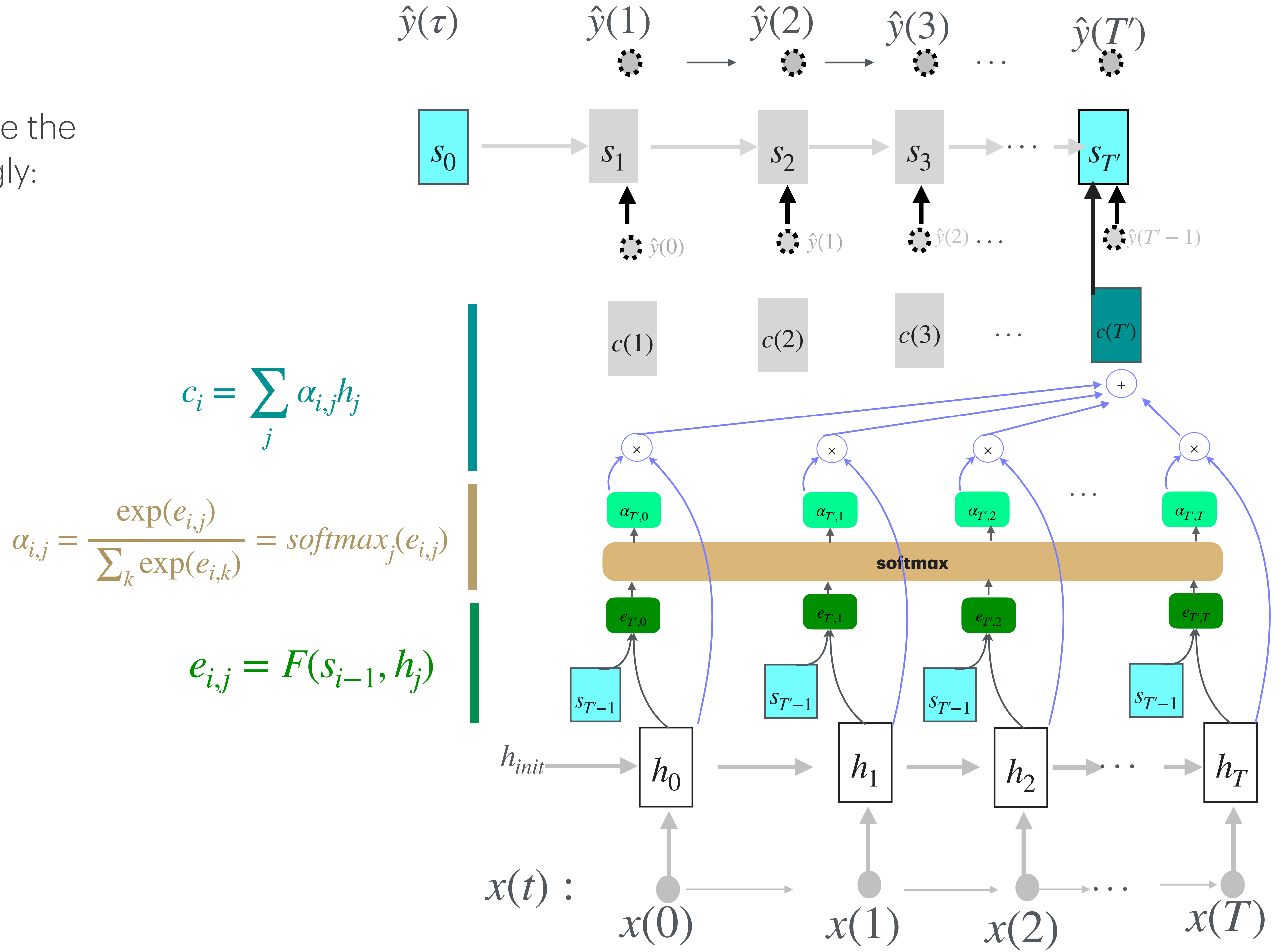
- Now we can update the diagram accordingly:



- Now we can update the diagram accordingly:



- Now we can update the diagram accordingly:



- Now we can update the diagram accordingly:

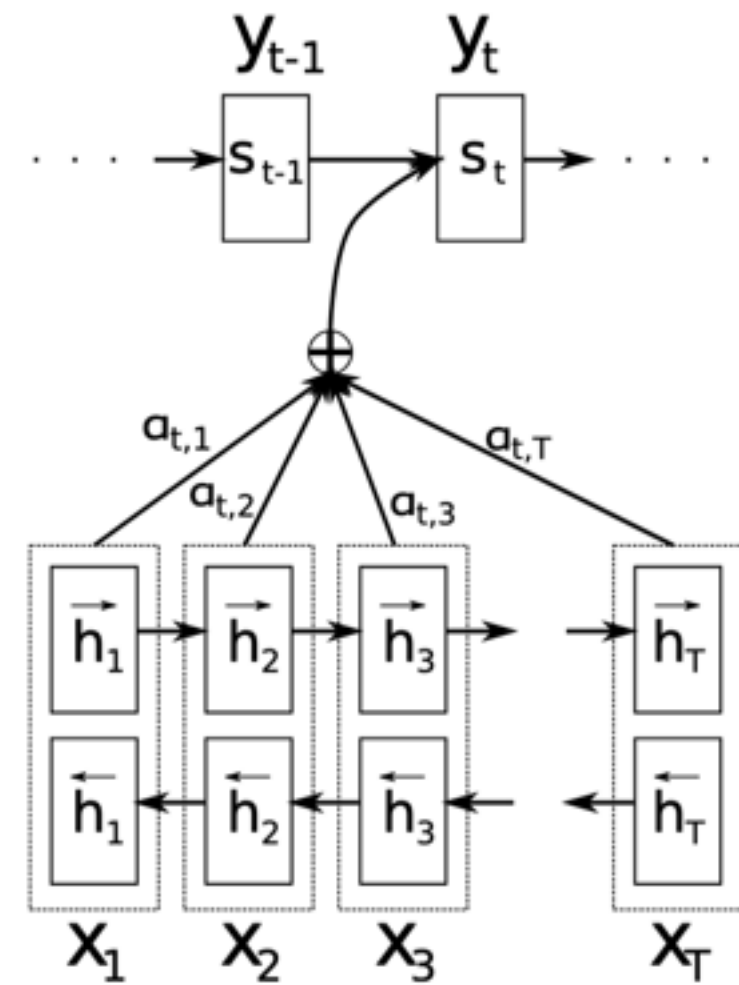
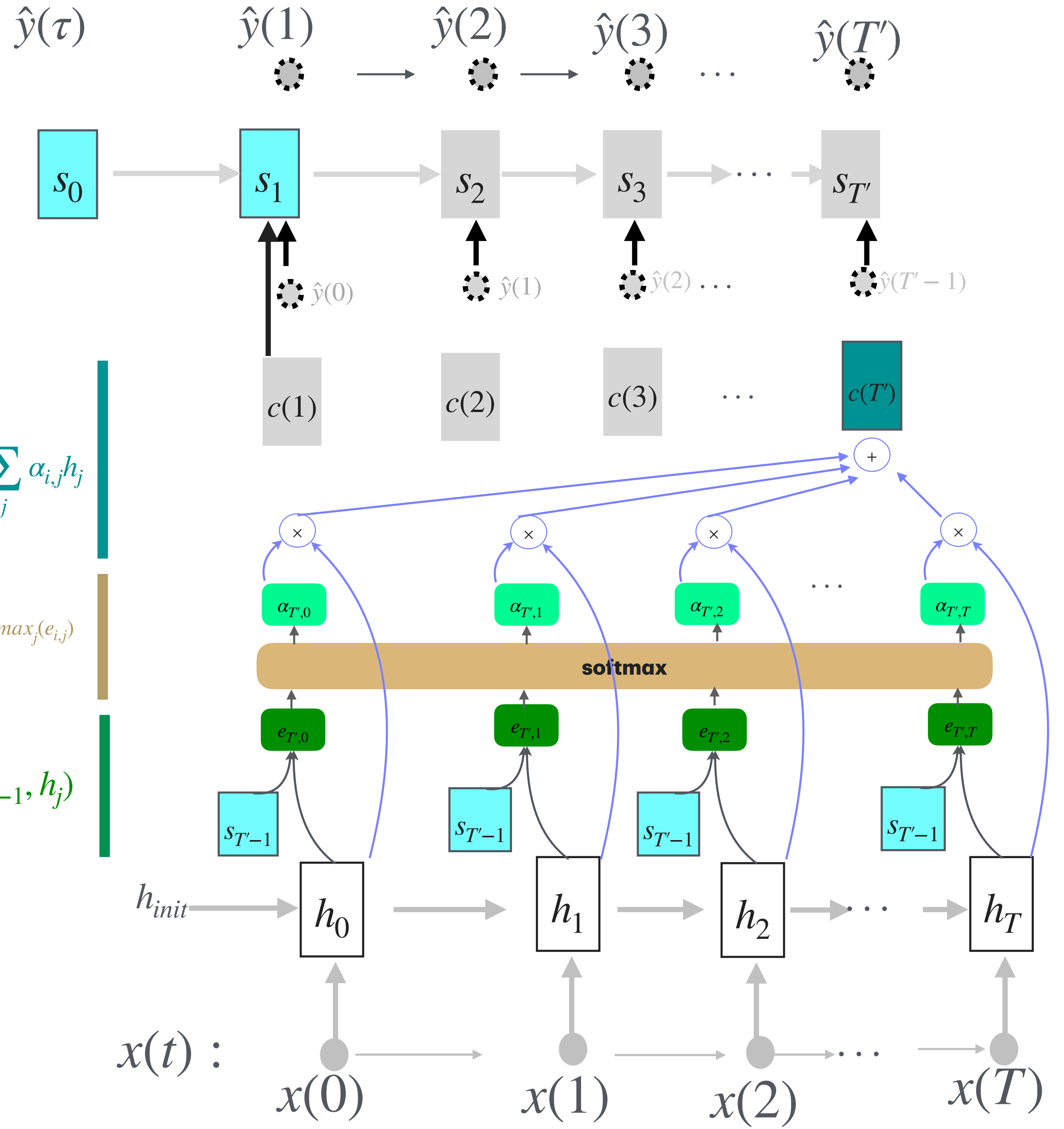


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

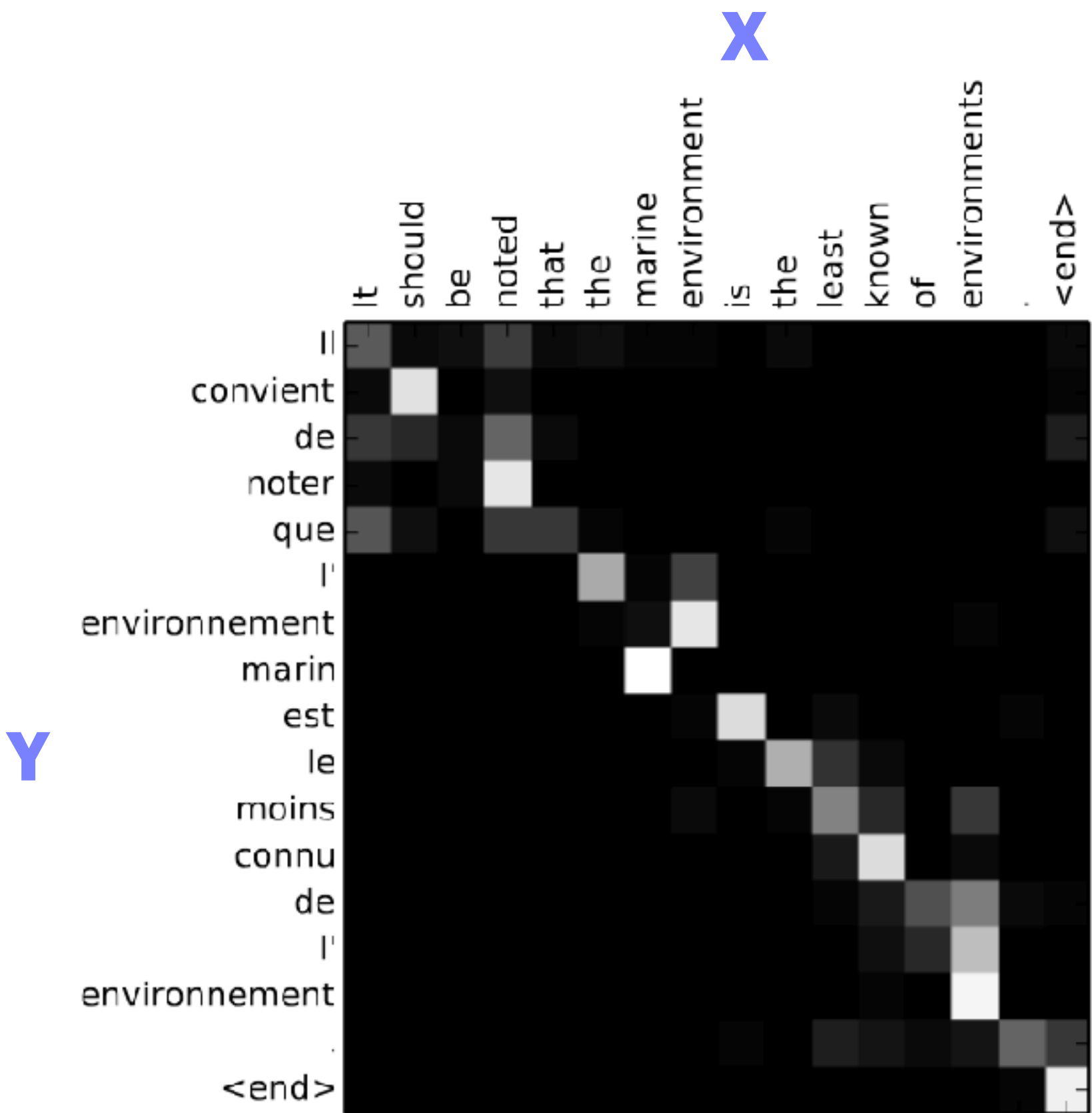
$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_k \exp(e_{i,k})} = \text{softmax}_j(e_{i,j})$$

$$e_{i,j} = F(s_{i-1}, h_j)$$

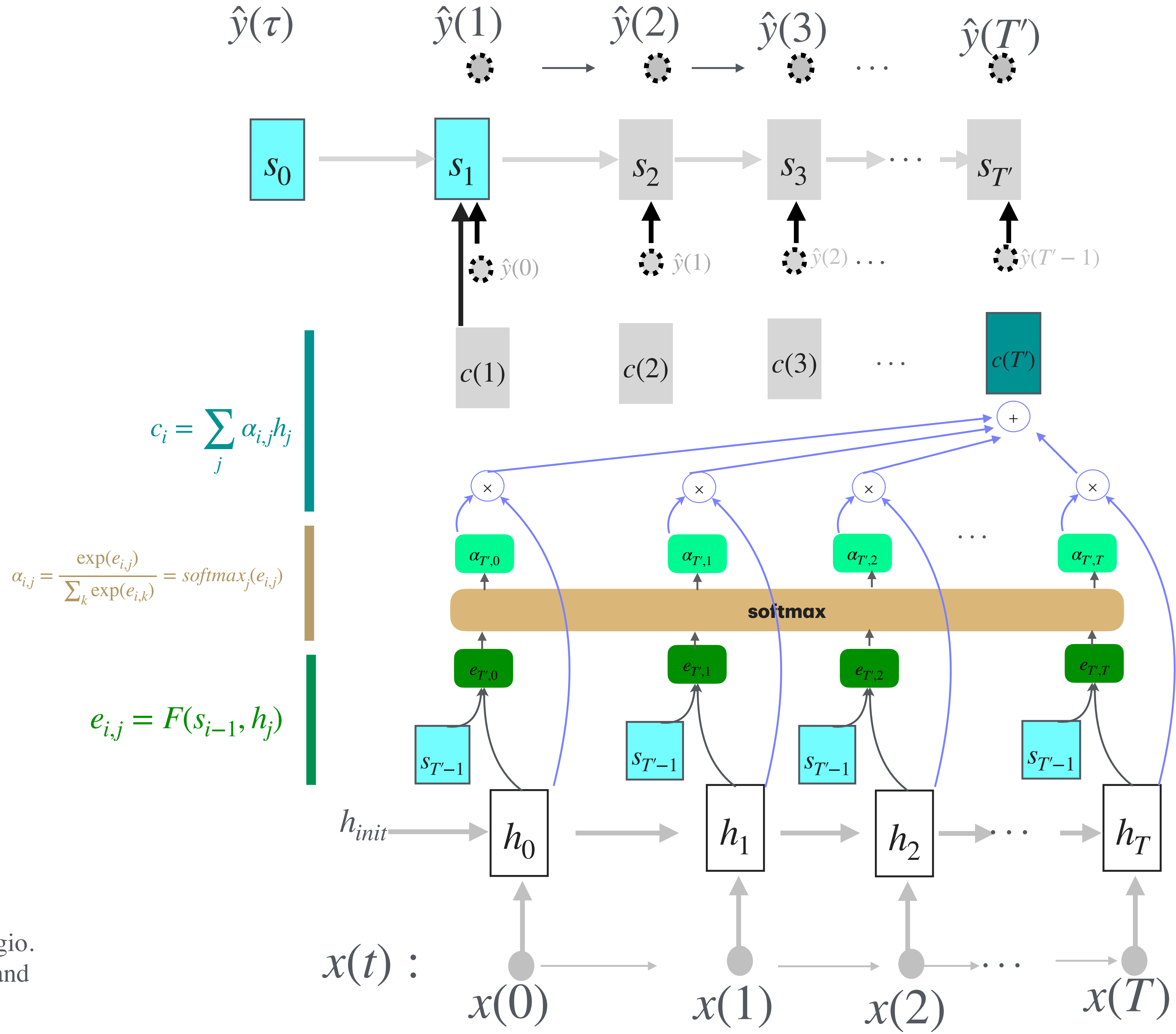
$$c_i = \sum_j \alpha_{i,j} h_j$$



- Now we can update the diagram accordingly:



Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." *arXiv preprint arXiv:1409.0473* (2014).



Recipe Card: Autoregressive Seq2Seq + Attention

Type of Learning: Supervised + Autoregressive

Data set: Input sequence + output sequence

Data

- 1- Load libraries: Basics + PyTorch
- 2-Load your sequence dataset **D**:
 - Same as Autoregressive Recurrent Models (seq2seq)
- 3- Now set up the DataLoader for training/validation/testing.
 - Same as **Autoregressive Recurrent Models** (seq2seq)

NN model

- 4-Build the *autoregressive seq2seq + attention model* (It has 4 parts):

A) Encoder (choose nn.RNN(...) nn.GRU(...) nn.LSTM(...))

$$x \rightarrow [\text{RNN / GRU / LSTM encoder}] \rightarrow H$$

-Input: $x \in [B, T, F]$. output is (Unlike seq2seq, we keep all encoder hidden states):

$$H = [h_1, \dots, h_{T_x}], \quad H \in [B, T_x, d_h],$$

B) Attention module: At decoder step τ , calculate the context vector c_τ :

- 1) relevance score**: $e_{\tau,j} = F(s_{\tau-1}, h_j)$
- 2) attention weights: $\alpha_{\tau,j} = \text{softmax}_j(e_{\tau,j})$
- 3) context vector for decoder step τ : $c_\tau = \sum_j \alpha_{\tau,j} h_j$

**note: the function F in PyTorch could be a small linear based module: nn.Linear(...)

C) Decoder Generate one output step at a time using previous decoder state, previous output, and current context:

$$s_\tau = f_{dec}(s_{\tau-1}, \hat{y}_{\tau-1}, c_\tau)$$

D) Output head:

$$s_\tau \rightarrow [\text{Linear layer}] \rightarrow out, \hat{y}_\tau$$

So, overall:

$$x_1, \dots, x_{T_x} \rightarrow \text{Encoder} \rightarrow H \rightarrow \text{Attention at each decoder step} \rightarrow c_1, c_2, \dots \rightarrow \hat{y}_1, \hat{y}_2, \dots, \hat{y}_{T_y}$$

Training

- 5-Define a proper loss function based on Y

a-If Y is continuous, then use MSELoss or L1Loss

b-If Y is categorical, CrossEntropyLoss on raw logits

c-For autoregressive part:

- 1) compute loss over all relevant *decoder* steps
- 2) ignore padded positions using masking if needed
- 3) total loss = sum or mean over decoder steps

-Add regularization to prevent overfitting

-The loss will be the sum of all relevant losses

- 6-Define the train and test functions:

The details of forward rollout:

a-Encoder reads the input sequence and outputs all encoder states context $H = [h_1, \dots, h_{T_x}]$

b-At each decoder step τ calculate the context c_τ

c-update decoder state

d-generate output $\hat{y}(\tau)$

training: teacher forcing can be used by feeding true previous target to model $y(\tau - 1)$

Inference: use model's previous prediction $\hat{y}(\tau - 1)$

- 7-Define training and run the training:

-Same as standard recurrent network card

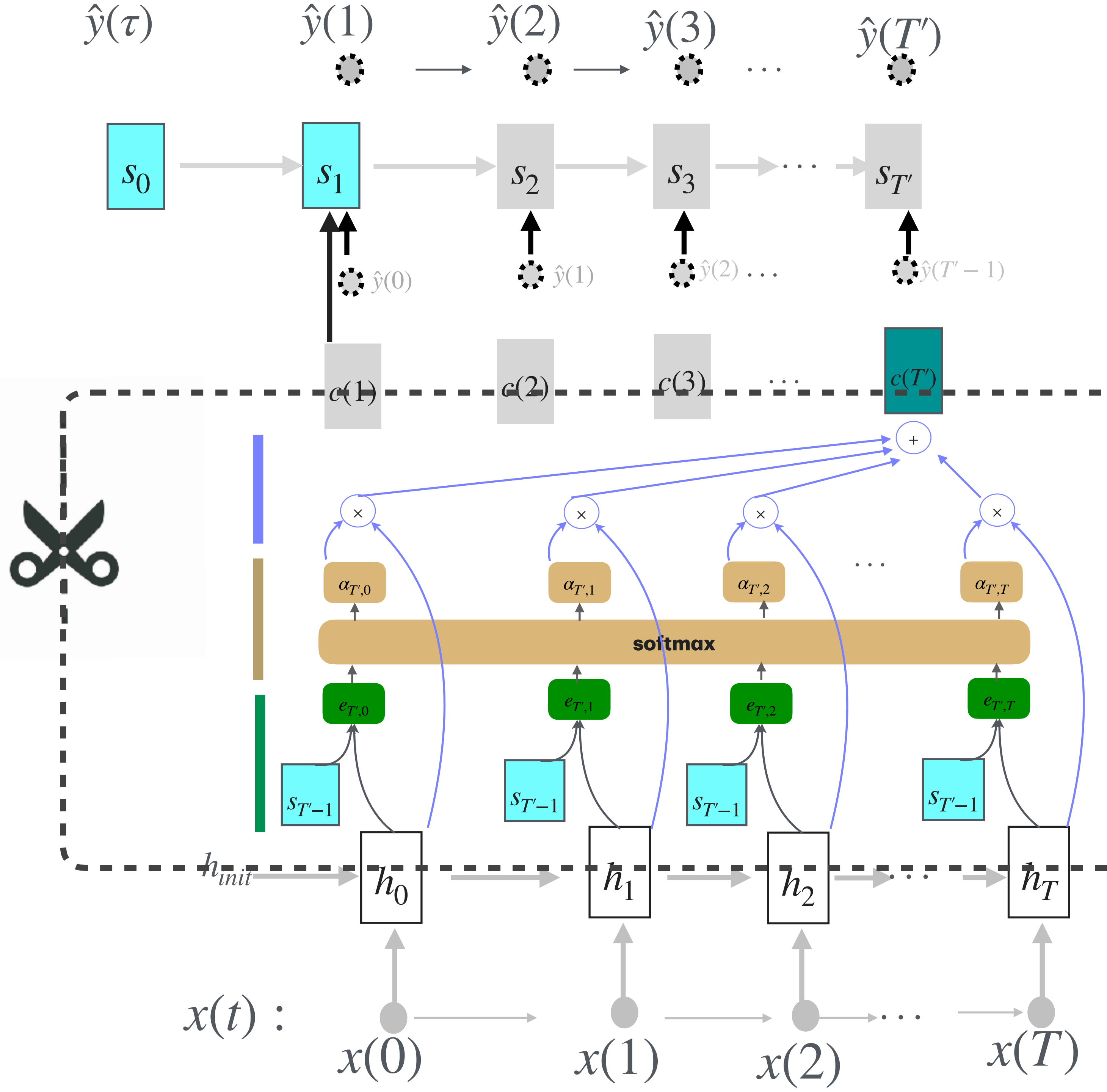
Part II: Attention as a standalone building block

Attention: As an operator

Attention lets each decoder step read from the encoder states:

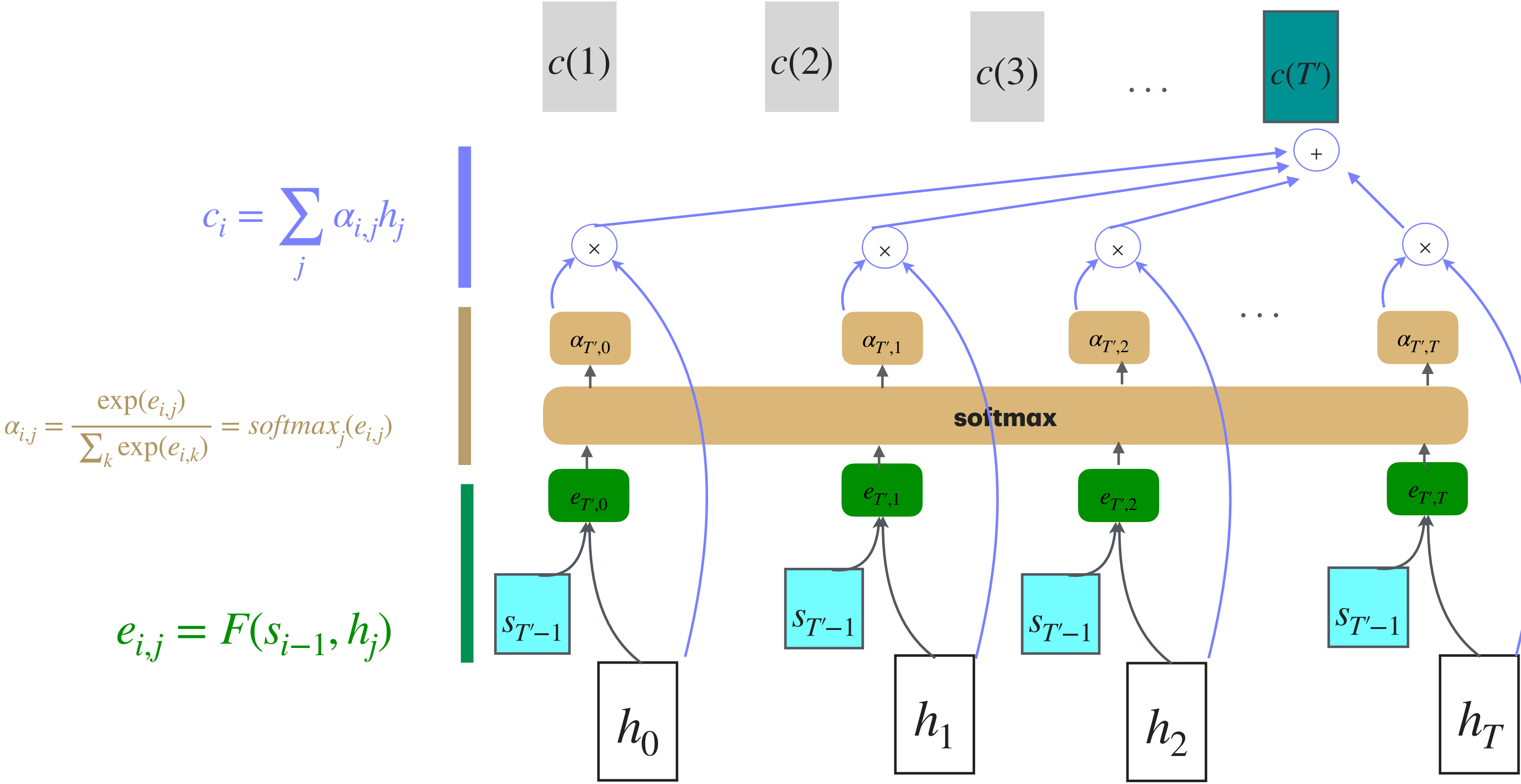
score $\rightarrow$ softmax $\rightarrow$ weighted sum

People realized that this operation does not require an RNN and it can be generalized.



Attention: **As an operator**

- Let's take this operation, and look at it carefully step by step:



Attention: **As an operator**

Step 1

We have a set of stored representations, call them **Keys (K)**
 $k_j \in \{k_0, k_1, \dots, k_T\}$ that encode the information we want to retrieve from.
In seq2seq setting, they are encoder states $k_j = h_j$

$k_0 = h_0$

$k_1 = h_1$

$k_2 = h_2$

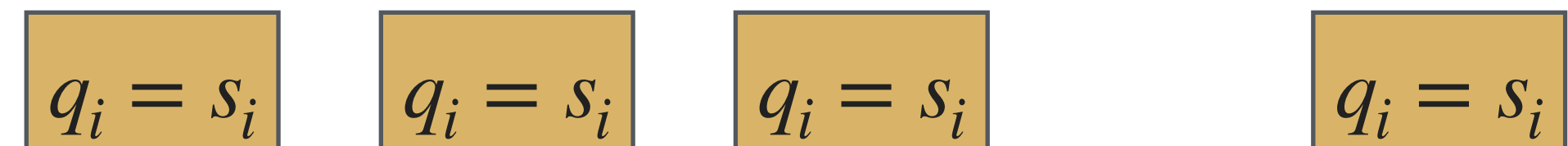
...

$k_T = h_T$

Step 2

We also have a **Query (Q)**, q_i a vector representing what we are looking for.

In seq2seq setting, query is the decoder hidden $q_i = s_i$



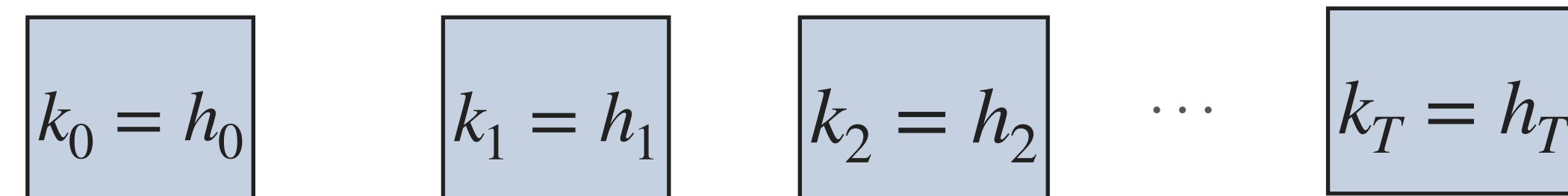
The diagram consists of four yellow rectangular boxes with black borders, arranged horizontally. Each box contains the mathematical expression $q_i = s_i$.

Step 1

We have a set of stored representations, call them **Keys (K)**

$k_j \in \{k_0, k_1, \dots, k_T\}$ that encode the information we want to retrieve from.

In seq2seq setting, they are encoder states $k_j = h_j$



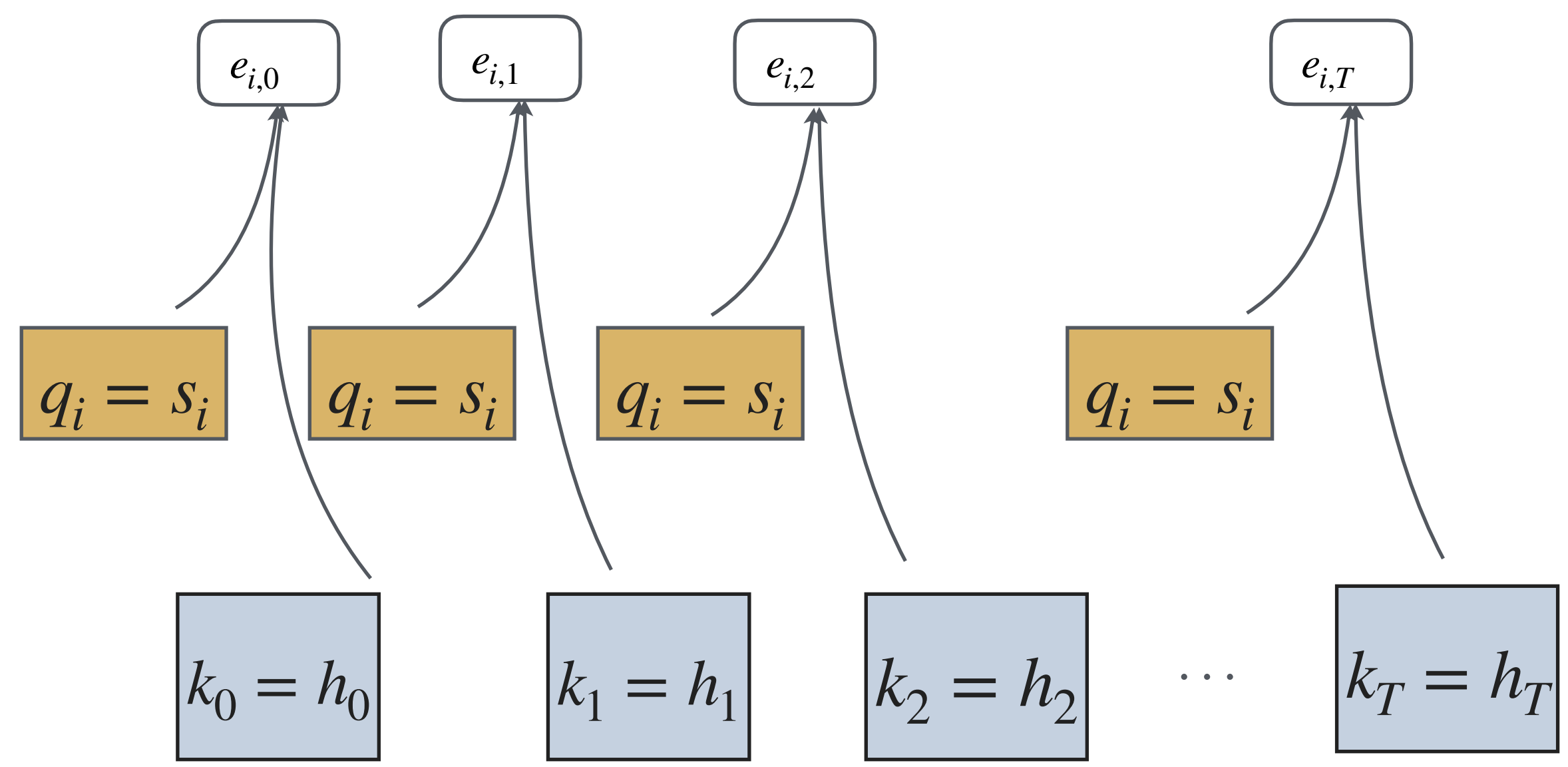
The diagram shows a sequence of blue rectangular boxes with black borders. The first three boxes contain the expressions $k_0 = h_0$, $k_1 = h_1$, and $k_2 = h_2$ respectively. This is followed by an ellipsis ($\dots$), and then a final box containing $k_T = h_T$.

Step 3

Now compare the current Query q_i with each Keys $k_j \in \{k_0, k_1, \dots, k_T\}$.
gives raw compatibility scores: $e_{i,j} = F(q_i, k_j)$

Step 2
We also have a **Query (Q)**, q_i a vector representing what we are looking for.
In seq2seq setting, query is the decoder hidden $q_i = s_i$

Step 1
We have a set of stored representations, call them **Keys (K)**
 $k_j \in \{k_0, k_1, \dots, k_T\}$ that encode the information we want to retrieve from.
In seq2seq setting, they are encoder states $k_j = h_j$



Step 4

Now apply softmax to the raw scores and get attention weights:

$$\alpha_{i,j} = \text{softmax}_j(e_{i,j})$$

Step 3

Now compare the current Query q_i with each Keys $k_j \in \{k_0, k_1, \dots, k_T\}$.

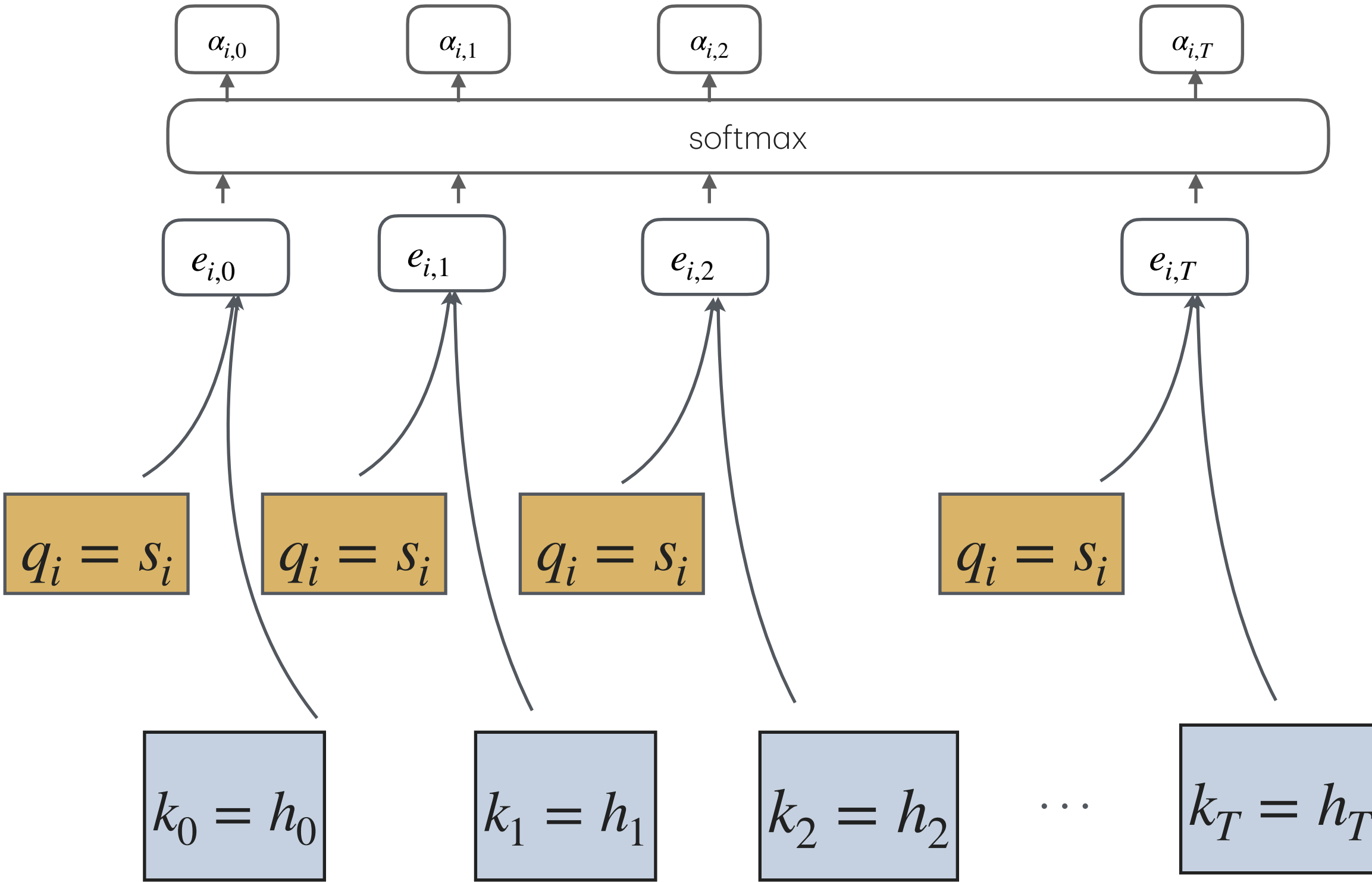
gives raw compatibility scores: $e_{i,j} = F(q_i, k_j)$

Step 2

We also have a **Query (Q)**, q_i a vector representing what we are looking for.
In seq2seq setting, query is the decoder hidden $q_i = s_i$

Step 1

We have a set of stored representations, call them **Keys (K)**
 $k_j \in \{k_0, k_1, \dots, k_T\}$ that encode the information we want to retrieve from.
In seq2seq setting, they are encoder states $k_j = h_j$



Step5

Now use the attention weights $\alpha_{i,j}$ to combine the corresponding **Values (V)** and form the final output for the current Query:

$$c_i = \sum_j \alpha_{i,j} v_j$$

In this original seq2seq setting, the same stored vectors play both roles:

$$v_j = h_j$$

Step 4

Now apply softmax to the raw scores and get attention weights:

$$\alpha_{i,j} = \text{softmax}_j(e_{i,j})$$

Step 3

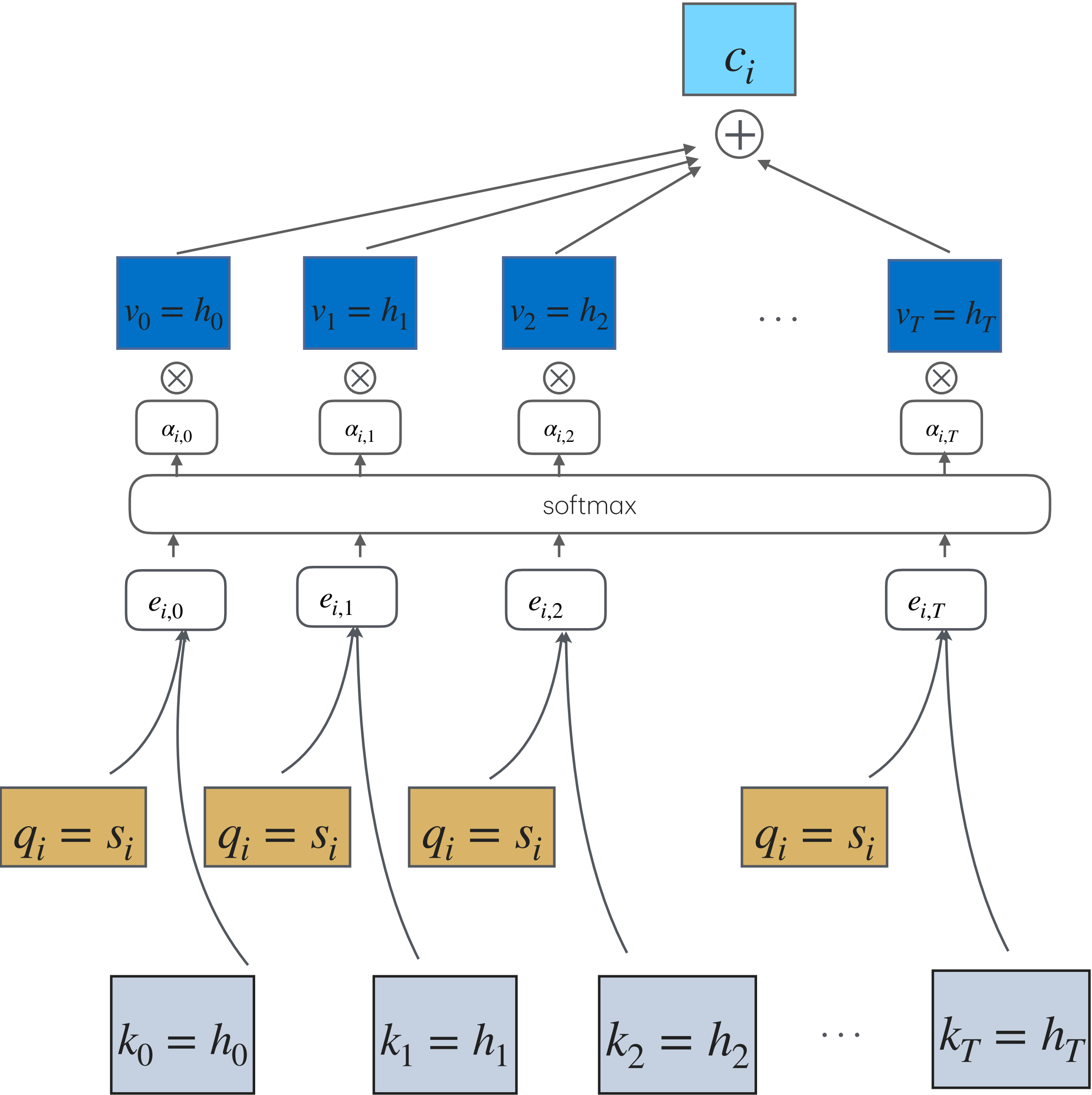
Now compare the current Query q_i with each Keys $k_j \in \{k_0, k_1, \dots, k_T\}$.
gives raw compatibility scores: $e_{i,j} = F(q_i, k_j)$

Step 2

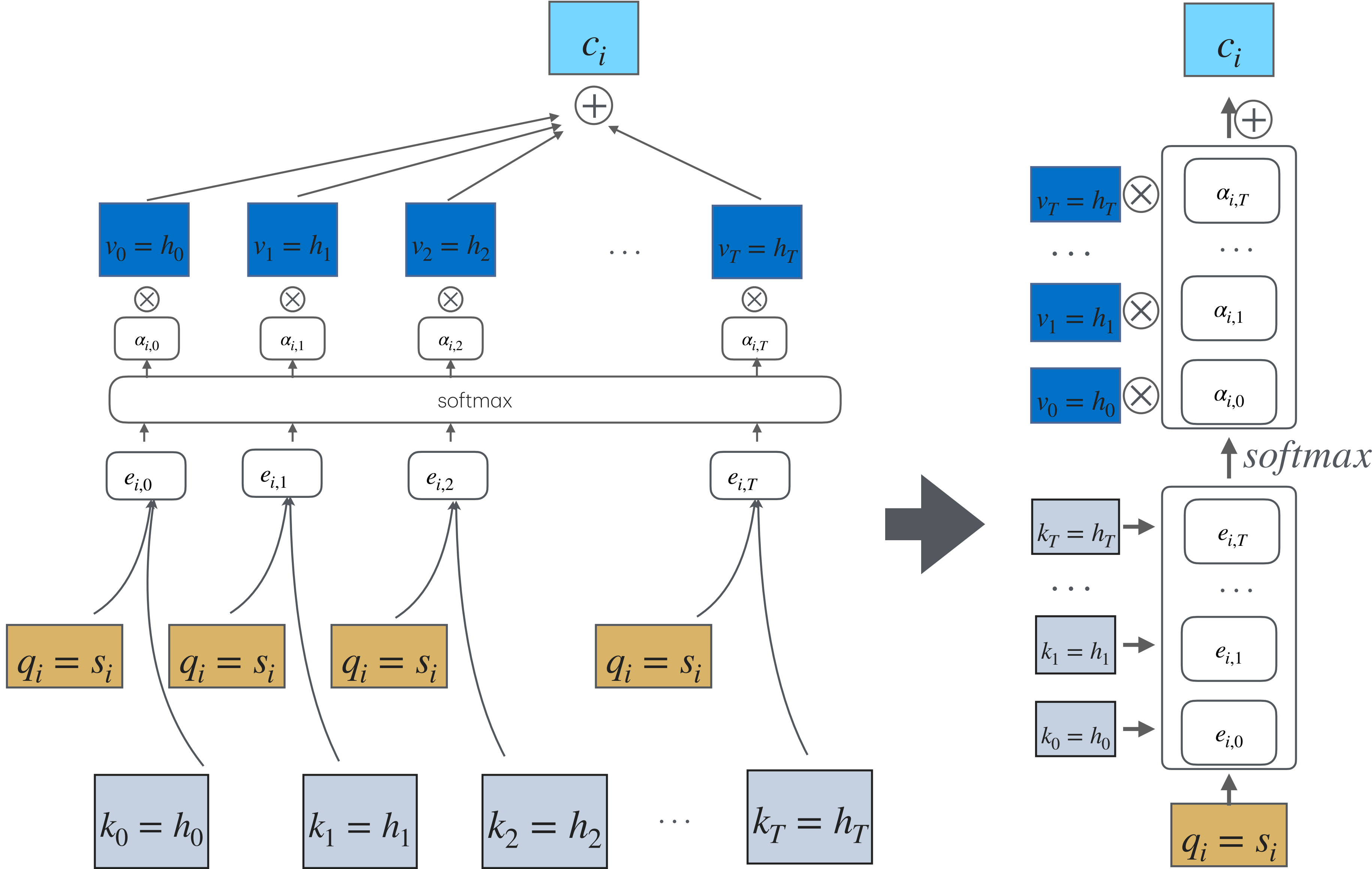
We also have a **Query (Q)**, q_i a vector representing what we are looking for.
In seq2seq setting, query is the decoder hidden $q_i = s_i$

Step 1

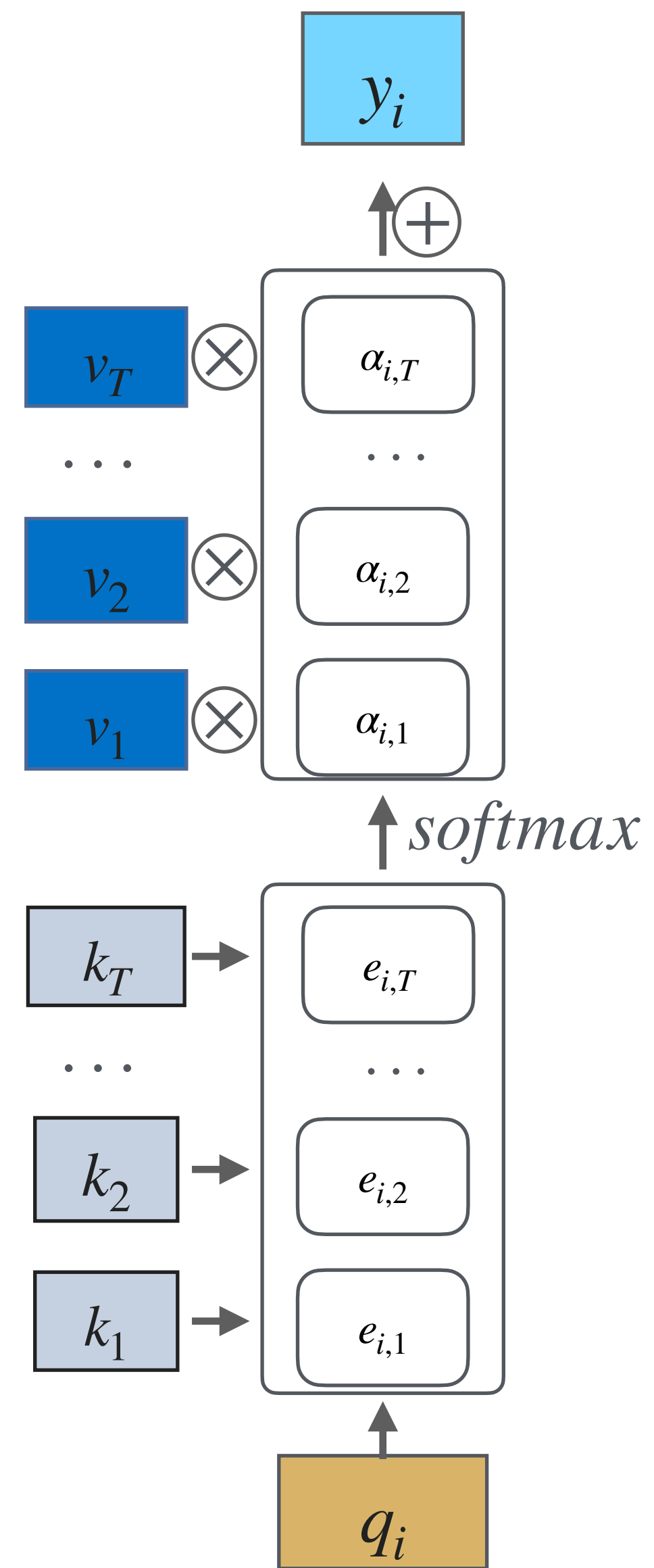
We have a set of stored representations, call them **Keys (K)**
 $k_j \in \{k_0, k_1, \dots, k_T\}$ that encode the information we want to retrieve from.
In seq2seq setting, they are encoder states $k_j = h_j$



We can now simplify the diagram, since we understand the roles of its components:



- Now let's rewrite of each step with all seq2seq references removed



- ***Note that I re-index everything to start from index 1, and not zero anymore.

- In seq2seq attention, we can view each encoder state h_j as both a key for matching and a value for retrieval:

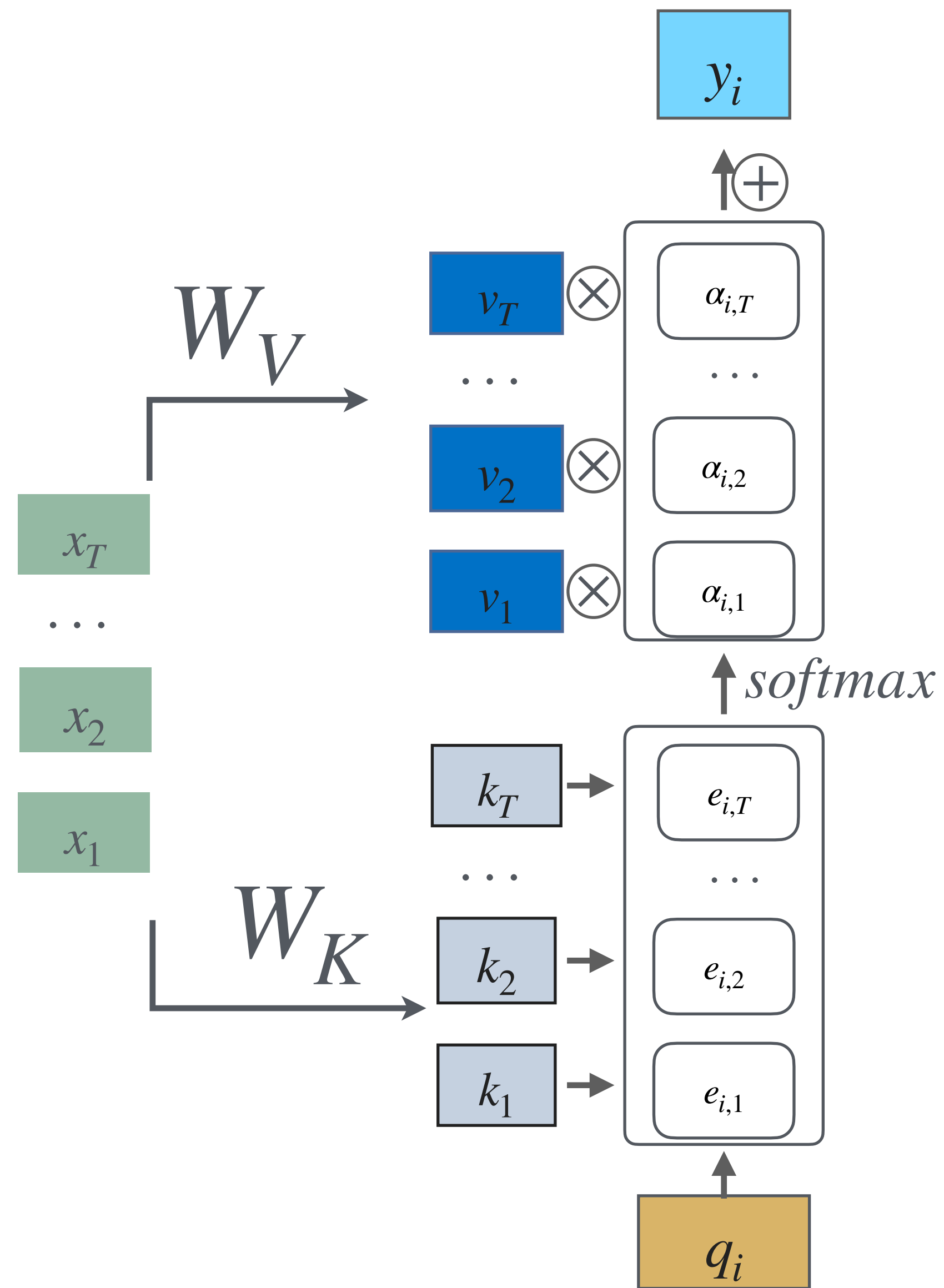
$$k_j = v_j = h_j$$

- More generally**, matching and retrieval can use different learned representations:

$$k_j = W_K x_j, \quad v_j = W_V x_j$$

- The query q_i is matched against the keys; the resulting weights combine the values:

$$\begin{aligned} e_{i,j} &= \text{score}(q_i, k_j) \\ \alpha_{i,j} &= \text{softmax}_j(e_{i,j}) \\ y_i &= \sum_j \alpha_{i,j} v_j \end{aligned}$$



Learn the Query:

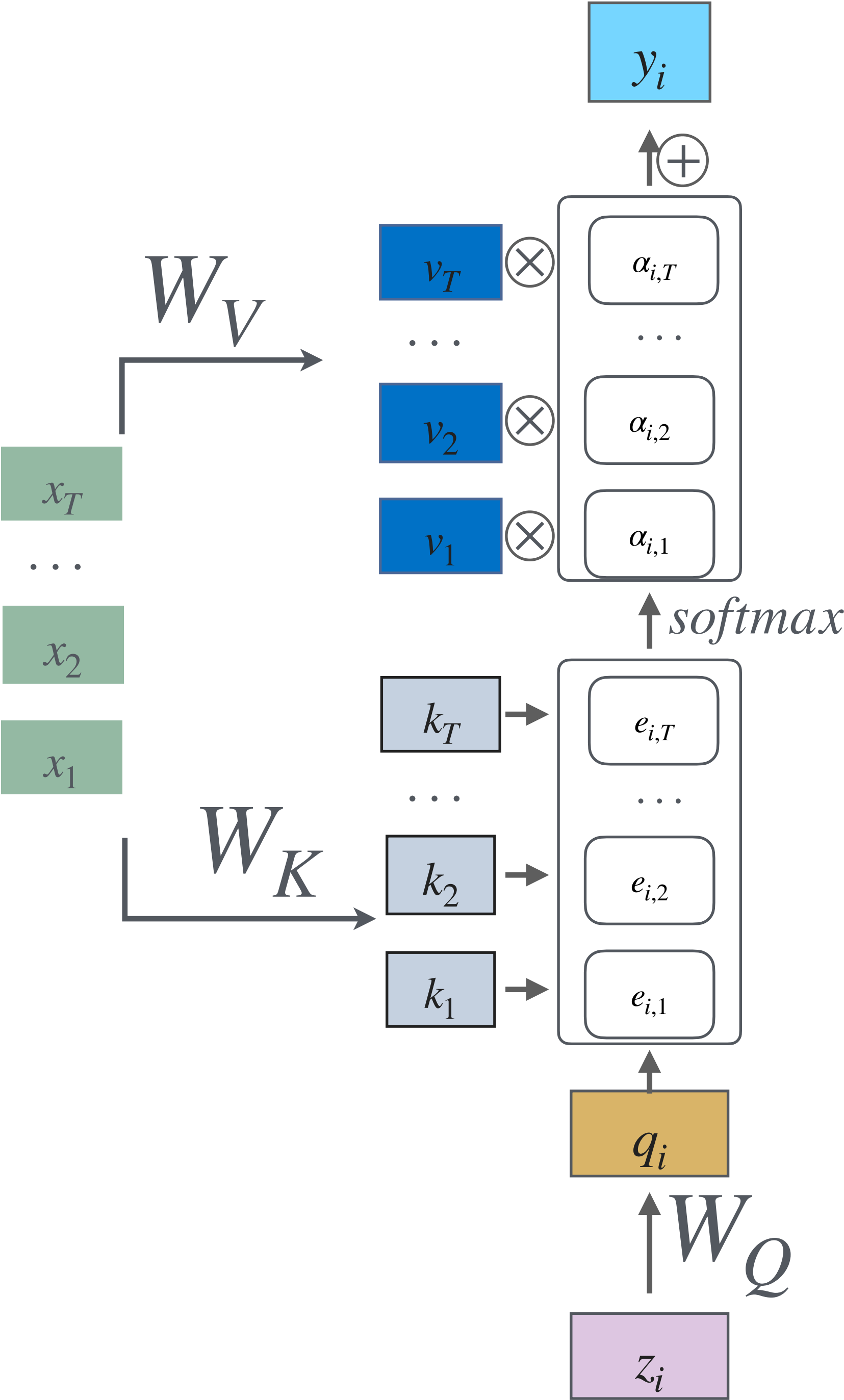
Let z_i be the representation currently seeking information:

Project it into a query:

$$q_i = W_Q z_i$$

Match this query against the keys:

$$e_{i,j} = \text{score}(q_i, k_j)$$



- So far, we computed attention for one query input z_i
- In a sequence model, every output position has its own query input:

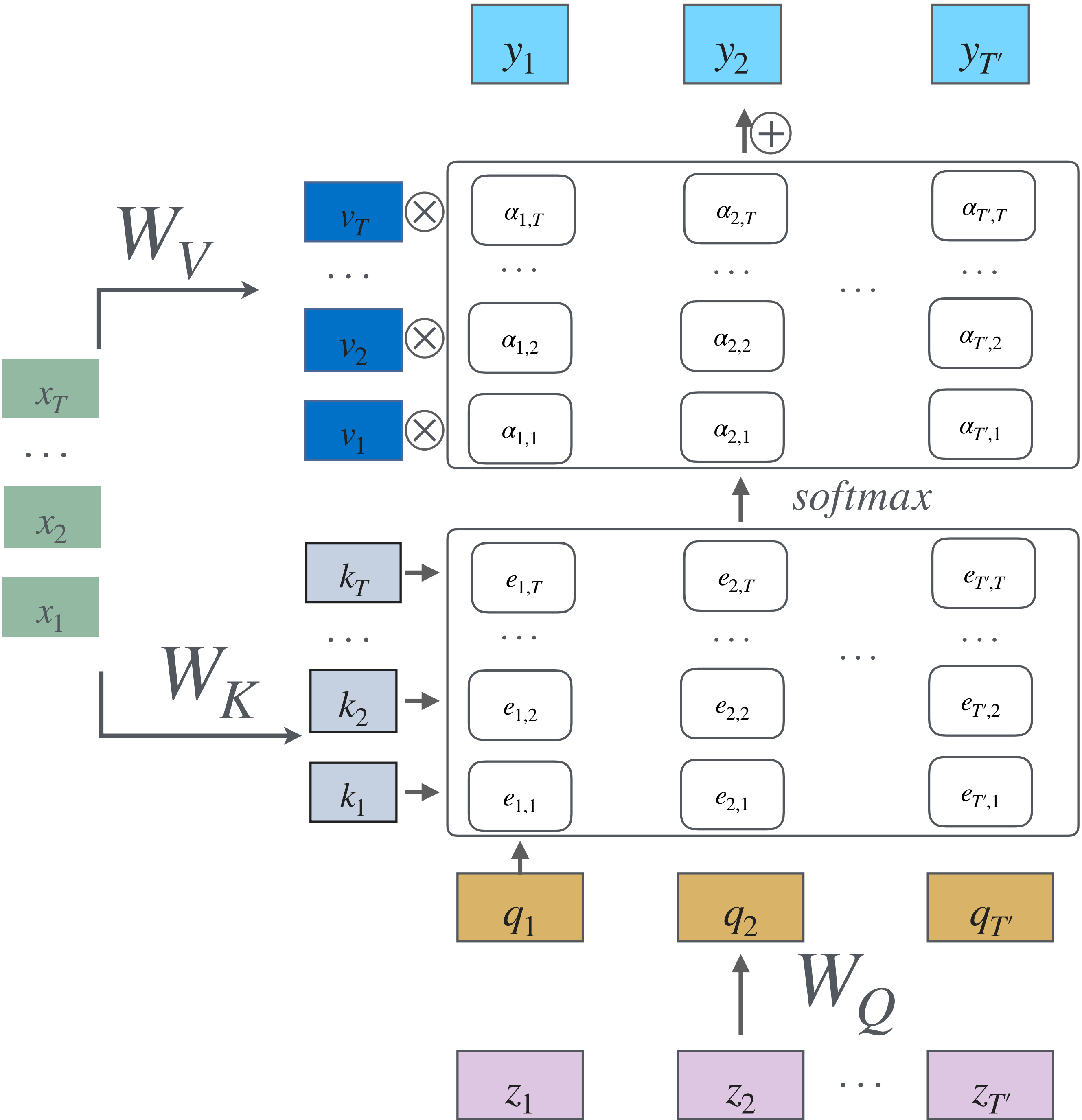
$$z_1, z_2, \dots, z_{T'}$$

Project each one into a query:

$$q_i = W_Q z_i$$

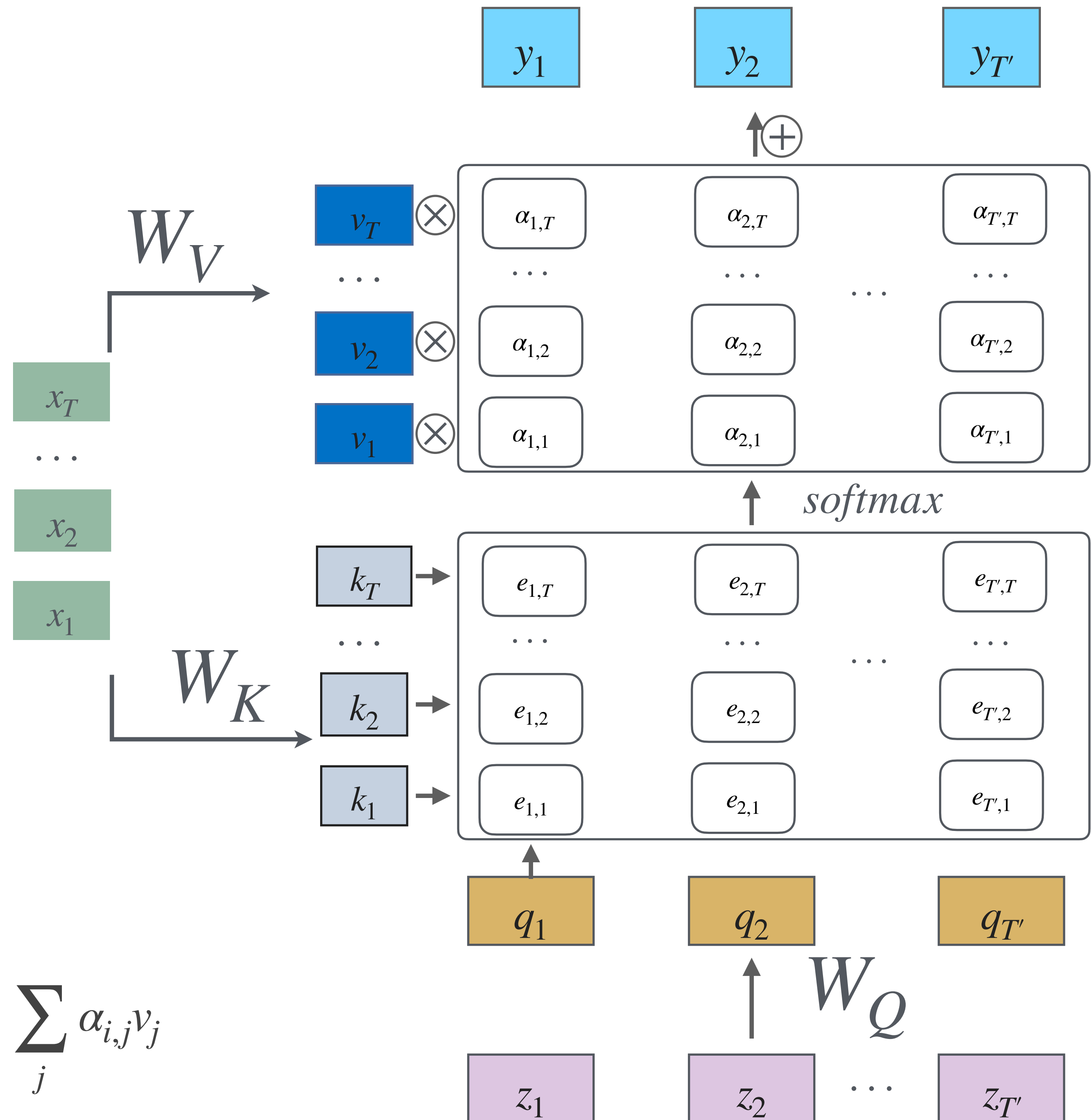
Each query attends over the same keys and values, producing its own output y_i

- ***Note that I re-index everything to start from index 1, and not zero anymore.



Attention in its general form:

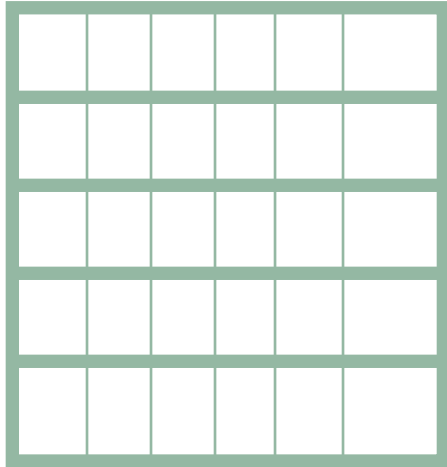
- Queries ask,
- Keys determine relevance
- Values are combined



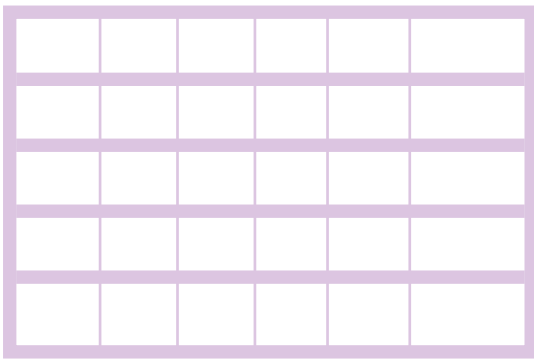
- Apply the same operation to every query q_i :

$$e_{i,j} = \frac{q_i^\top k_j}{\sqrt{d_k}} \quad \rightarrow \quad \alpha_{i,j} = \text{softmax}_j(e_{i,j}) \quad \rightarrow \quad y_i = \sum_j \alpha_{i,j} v_j$$

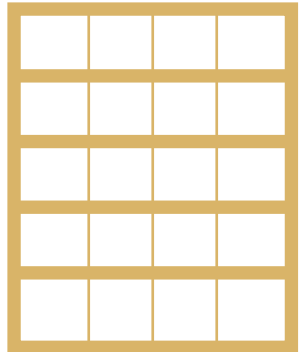
Rewriting Attention in matrix form

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix}$$


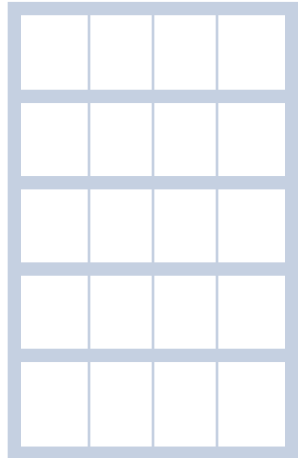
$T \times d_x$

$$Z = \begin{bmatrix} z_1^\top \\ \vdots \\ z_{T'}^\top \end{bmatrix}$$


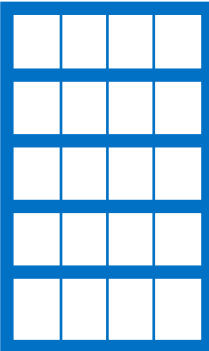
$T' \times d_z$

$$Q = ZW_Q$$


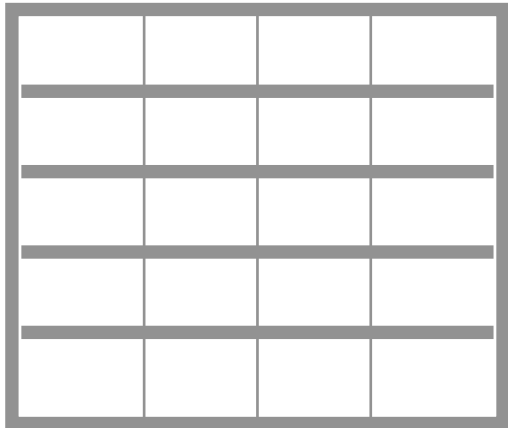
$T' \times d_k$

$$K = XW_K$$


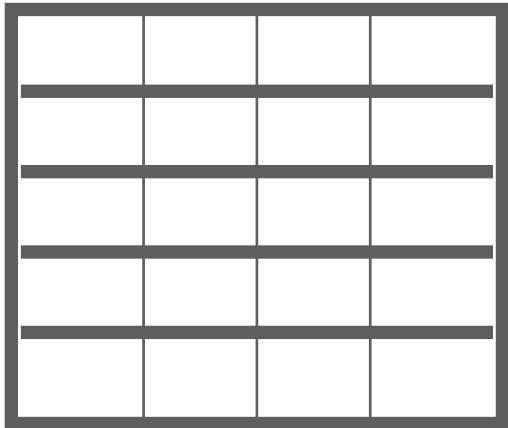
$T \times d_k$

$$V = XW_V$$


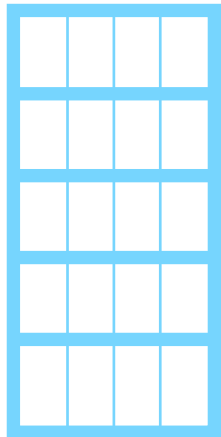
$T \times d_v$

$$E = \frac{QK^\top}{\sqrt{d_k}}$$


$T' \times T$

$$A = \text{softmax}_{\text{rowwise}}(E)$$


$T' \times T$

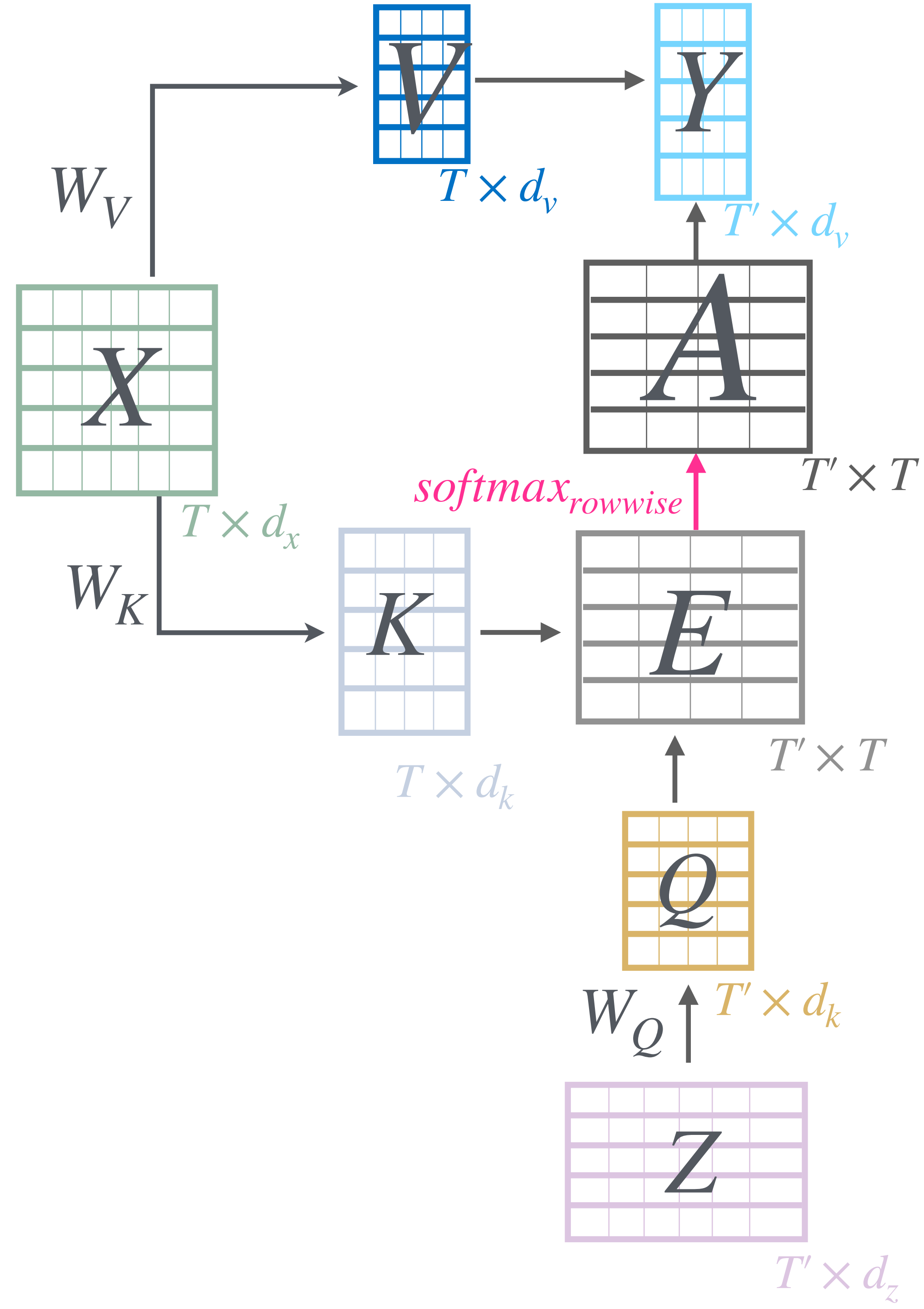
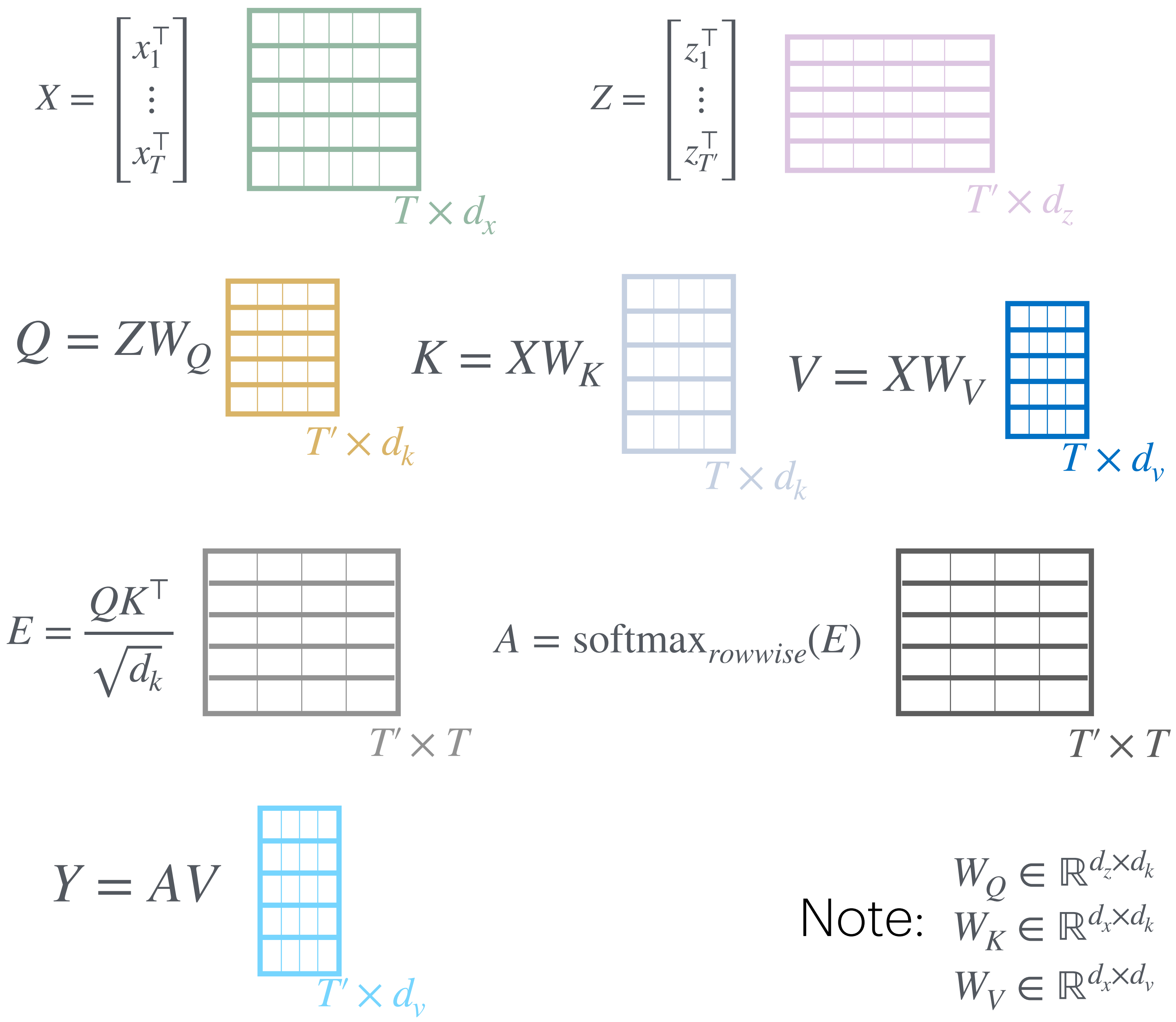
$$Y = AV$$


$T' \times d_v$

Note:

$$W_Q \in \mathbb{R}^{d_z \times d_k}$$
$$W_K \in \mathbb{R}^{d_x \times d_k}$$
$$W_V \in \mathbb{R}^{d_x \times d_v}$$

Rewriting Attention in matrix form



We can write it even more compactly:

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} \quad T \times d_x$$

$$Z = \begin{bmatrix} z_1^\top \\ \vdots \\ z_{T'}^\top \end{bmatrix} \quad T' \times d_z$$

$$Q = ZW_Q \quad T' \times d_k$$

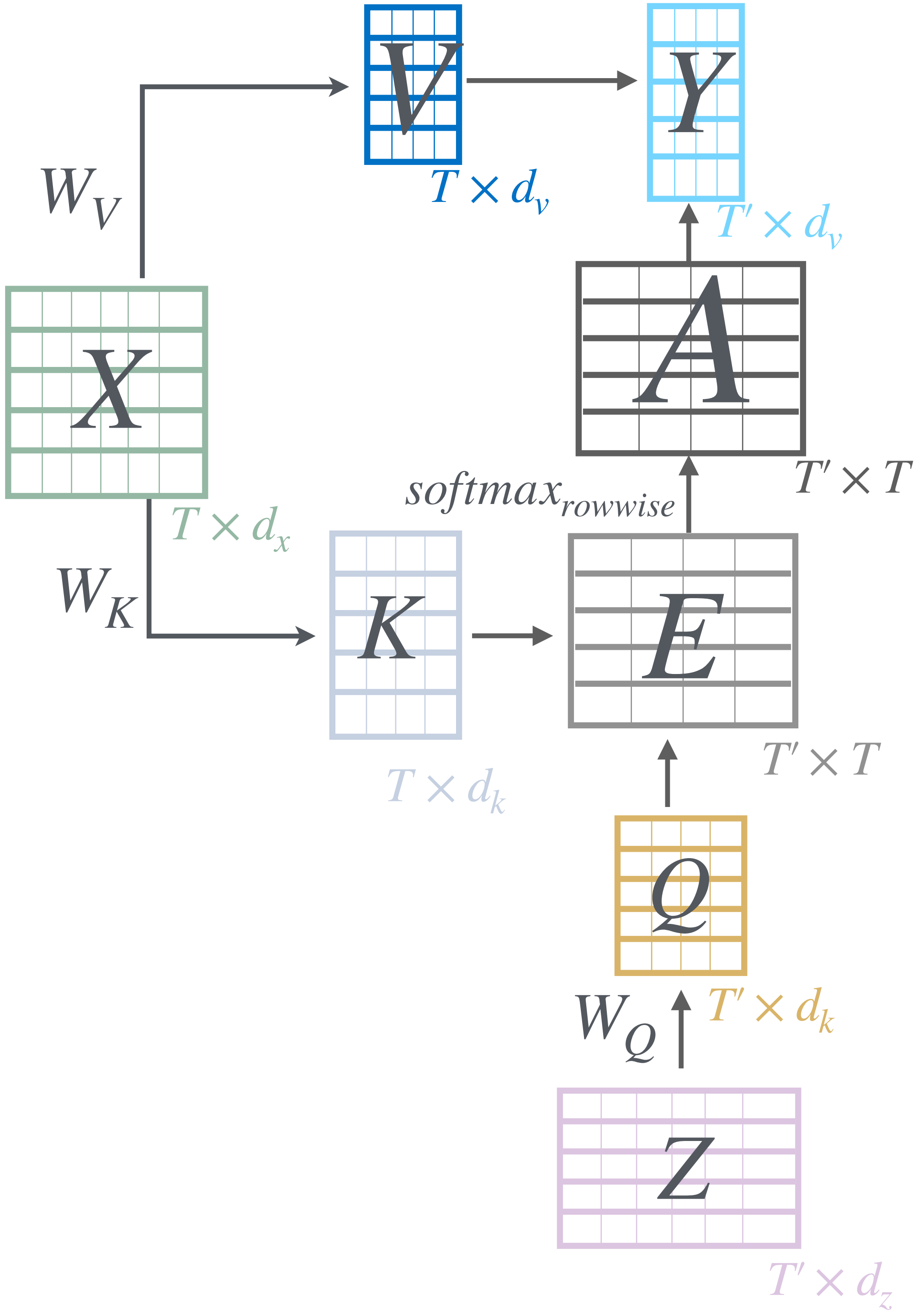
$$K = XW_K \quad T \times d_k$$

$$V = XW_V \quad T \times d_v$$

$$Y = \text{Attention}(Q, K, V) = \text{softmax}_{\text{rowwise}} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

Note:

$$W_Q \in \mathbb{R}^{d_z \times d_k}$$
$$W_K \in \mathbb{R}^{d_x \times d_k}$$
$$W_V \in \mathbb{R}^{d_x \times d_v}$$



Bonus

1) Attention is **permutation equivariant** in the Queries Z

Let P_Z permute the rows of Z . Then:

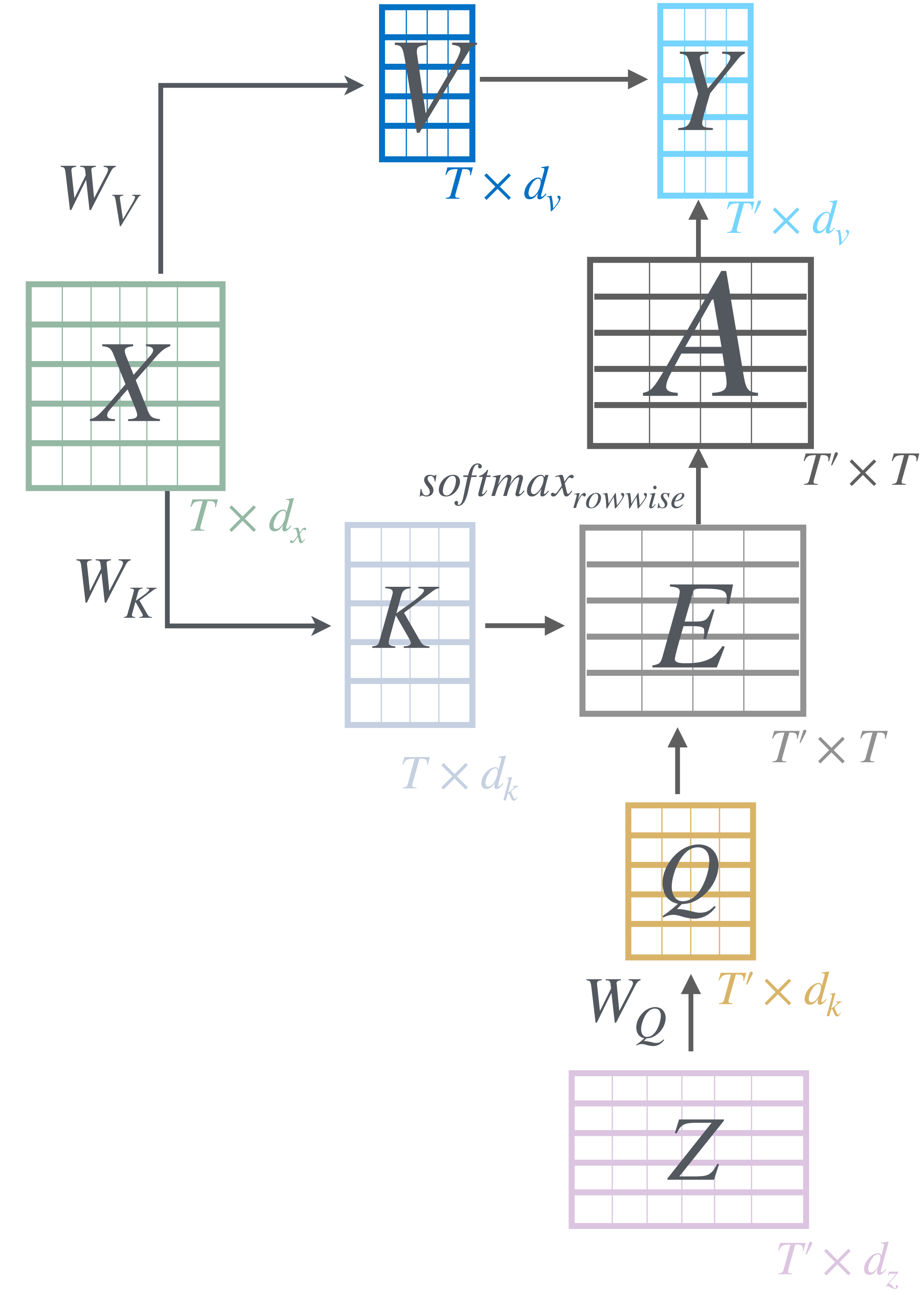
$$Z' = P_Z Z \quad \Rightarrow \quad Q' = P_Z Q \quad E' = \frac{Q' K^\top}{\sqrt{d_k}} = \frac{(P_Z Q) K^\top}{\sqrt{d_k}} = P_Z E$$

$$A' = \text{softmax}_{\text{rowwise}}(E') = \text{softmax}_{\text{rowwise}}(P_Z E) = P_Z A$$

$$Y' = A' V = (P_Z A) V = P_Z (A V) = P_Z Y$$

$$Y' = \text{Attention}(P_Z Q, K, V) = P_Z Y$$

Interpretation: Reordering the Queries reorders the outputs in the same way. That makes sense because each Query asks its own question, so changing their order should only change the order of the answers.



Bonus

2) Attention is **permutation invariant** in the Key/Value set X

Let P_X permute the rows of X . Then:

$$X' = P_X X \Rightarrow K' = P_X K, \quad V' = P_X V$$

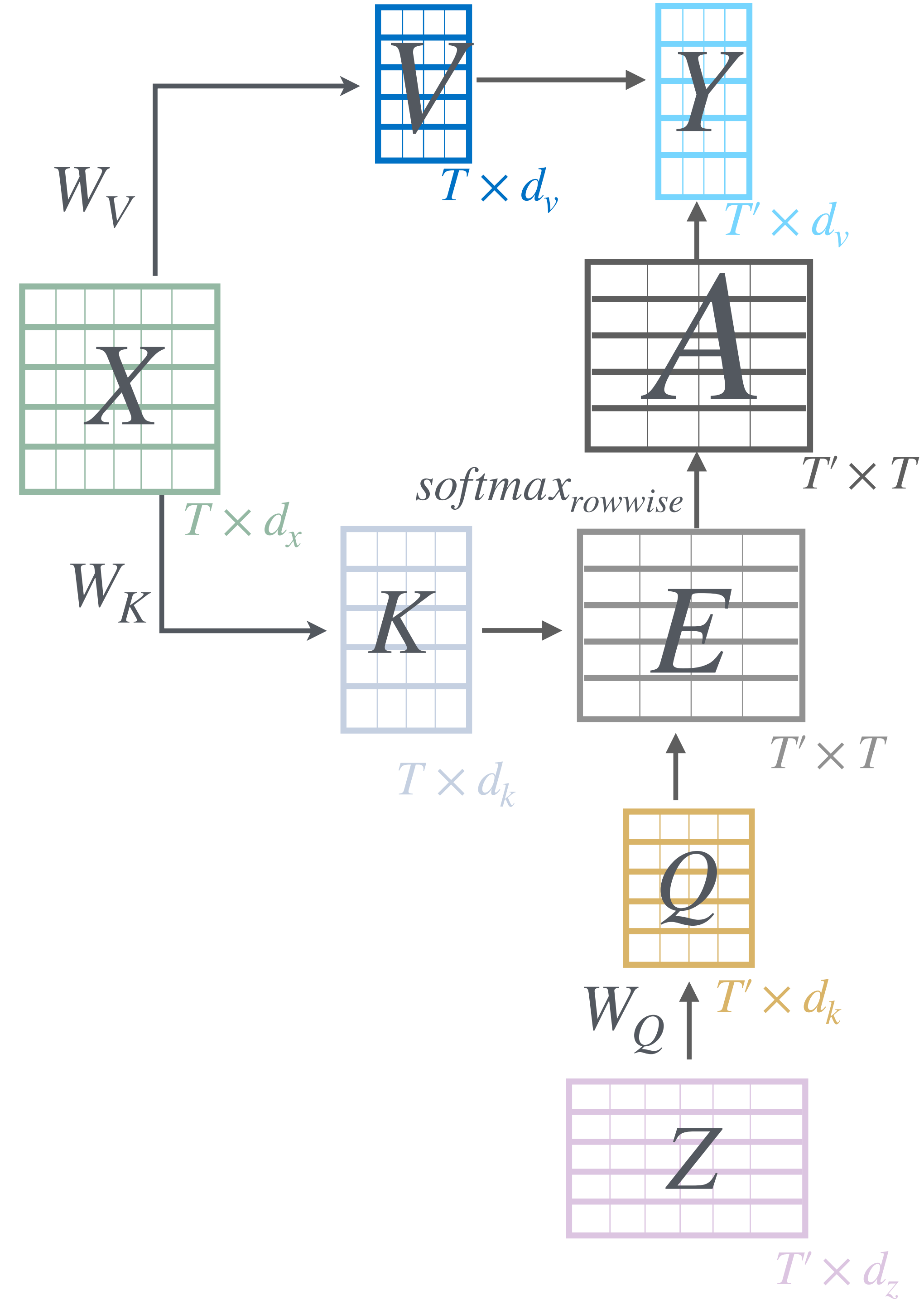
$$E' = \frac{Q(K')^\top}{\sqrt{d_k}} = \frac{Q(P_X K)^\top}{\sqrt{d_k}} = \frac{QK^\top P_X^\top}{\sqrt{d_k}} = EP_X^\top$$

$$A' = \text{softmax}_{\text{rowwise}}(E') = \text{softmax}_{\text{rowwise}}(EP_X^\top) = AP_X^\top$$

$$Y' = A'V' = (AP_X^\top)(P_X V) = A(P_X^\top P_X)V = AV = Y$$

$$Y' = \text{Attention}(Q, P_X K, P_X V) = Y$$

Interpretation: Reordering the Key/Value set does not change the output, as long as Keys and Values are permuted together. That makes sense because they act like a memory bank: changing the order of stored items should not change what is retrieved.



Attention Operator updates each element of Z by letting it softly read from all elements of X

We start with two sets of representations:

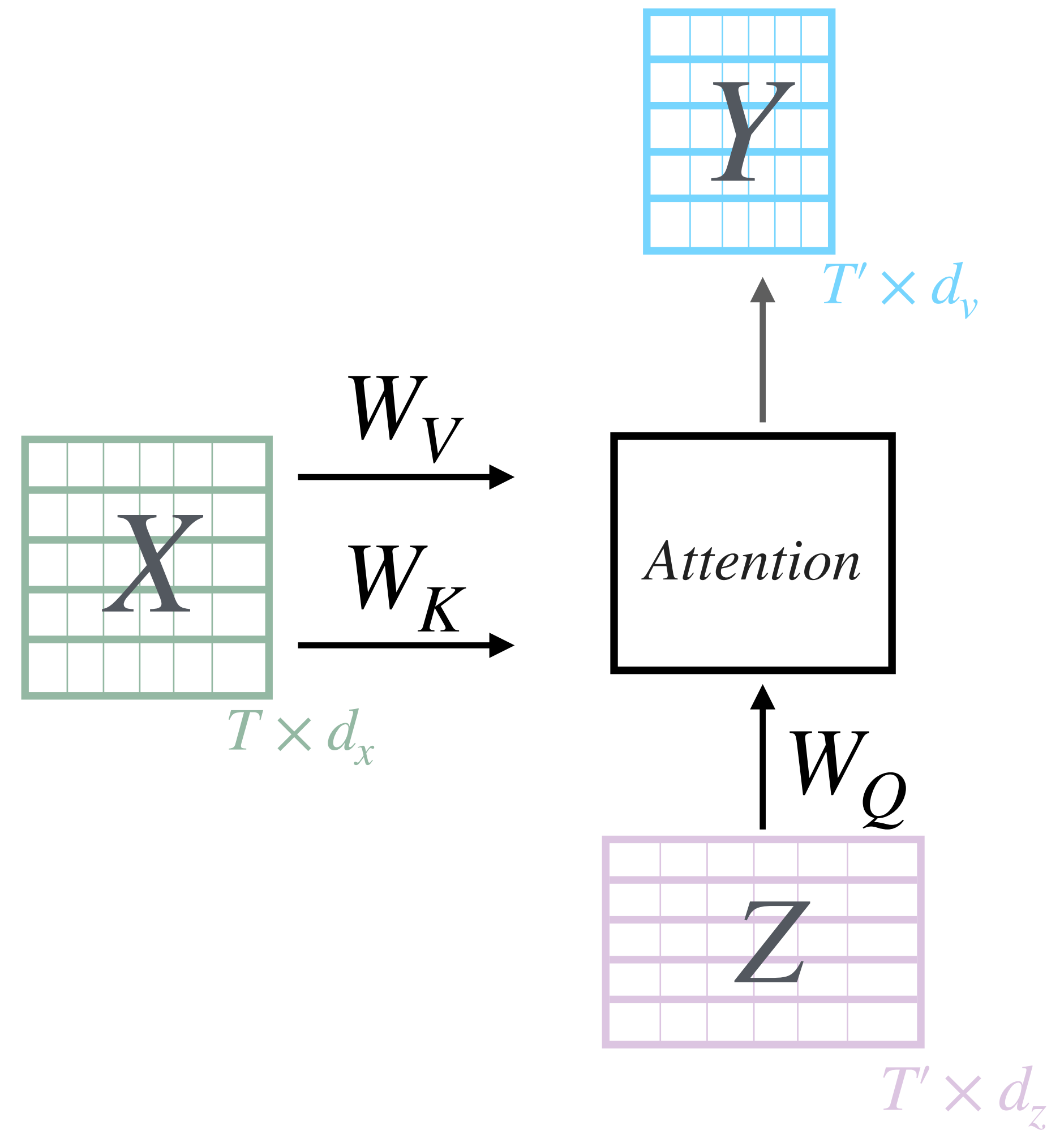
Z : containing T' query items (each has dim d_z).

X : containing T source items (each has dim d_x).

The idea of Attention is that each query in Z can retrieve information from the whole set X , not by selecting one item hard, but by forming a weighted combination of all T source items.

Doing this for all T' queries produces a new set of T' output representations Y .

The Attention operator implements this through learned projections W_Q , W_K , W_V followed by score computation, softmax normalization, and weighted mixing.



ingredient: **Attention Operator block (PyTorch implementation)**

Input/output

1- Input tensors

Query source: $Z \in [B, T', d_z]$

Key/Value source: $X \in [B, T, d_x]$

2- Output tensor:

Attention output: $Y \in [B, T', d_v]$

variable

1-Learnable projections: (Use nn.Linear(...)):

$W_Q : d_z \rightarrow d_k, W_K : d_x \rightarrow d_k, W_V : d_x \rightarrow d_v$

In PyTorch, where B is batch size:

$Q = W_Q(Z) \in [B, T', d_k]$

$K = W_K(X) \in [B, T, d_k]$

$V = W_V(X) \in [B, T, d_v]$

Computation

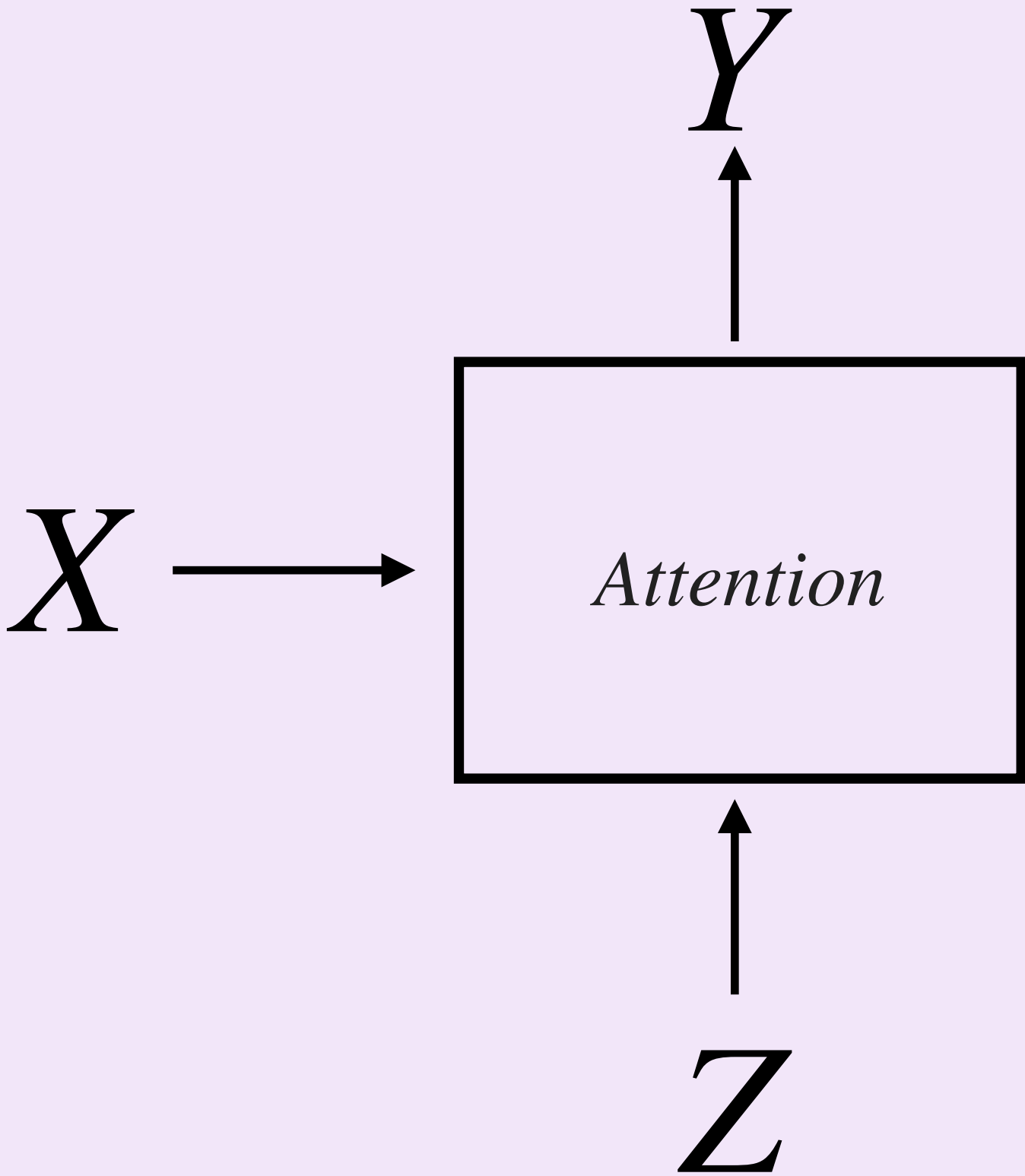
1-Attention computation:

$$E = \frac{QK^T}{\sqrt{d_k}} \in [B, T', T]$$

In PyTorch transpose Keys on the last two dims use
batched matmul:

$$A = \text{softmax}(E, \text{dim} = -1) \in [B, T', T]$$

2- $Y = AV \in [B, T', d_v]$



ingredient: **Attention Operator block (PyTorch implementation)**

Input/output

- 1- Input tensors
 - Query source: $Z \in [T', d_z]$
 - Key/Value source: $X \in [T, d_x]$
- 2- Output tensor:
 - Attention output: $Y \in [T', d_v]$

variable

- 1-Learnable projections: (Use nn.Linear(...)):
 - $W_Q : d_z \rightarrow d_k, W_K : d_x \rightarrow d_k, W_V : d_x \rightarrow d_v$
- In PyTorch, where B is batch size:
 - $Q = W_Q(Z) \in [T', d_k]$
 - $K = W_K(X) \in [T, d_k]$
 - $V = W_V(X) \in [T, d_v]$

Computation

- 1-Attention computation:
 - $E = \frac{QK^T}{\sqrt{d_k}} \in [T', T]$
 - In PyTorch transpose Keys on the last two dims use batched matmul:
 - $A = \text{softmax}(E, \text{dim} = -1) \in [T', T]$
- 2- $Y = AV \in [T', d_v]$

